代码随想录知识星球精华（最强八股文）第五版（算法篇）

代码随想录知识星球精华（最强八股文）第五版为九份PDF，分别是：

- 代码随想录知识星球八股文概述
- C++篇
- go篇
- Java篇
- 前端篇
- 算法题篇
- 计算机基础篇
- 问答精华篇
- 面经篇

本篇为最强八股文之算法篇。

题目解析

数组

数组中两个数的最大异或值

异或运算每一位单独运算（不进位加法），所以优先找二进制高位的异或结果为 1 的位置。

尽量让每一位的异或结果为 1 ，这样才是 num[i] ^ num[j]的最大值

保存前 i - 1 个数到字典树中，然后在字典树中查找与 i 匹配的异或结果最大值，保留。
遍历数组时，由于先将前一个数存入字典树中，然后再找当前数字在字典树中的异或最大值，所以遍历完所有数字之后，也就找到了最大的异或值。
在字典树中查找时，查找数字的每一位在字典树中相反的值，0 找 1 ，1 找 0 ，这样保证找到当前的异或最大值

找出所有子集的异或总和再求和

暴力可以直接过，直接枚举所有子集情况

(1) dfs递归的求出每个子集

求出异或结果和，并加起来
递归实现dfs，nums表示数组，i 表示枚举到的位置，xorSum表示已经累计的异或总和
递归内容，对于每个位置的元素都有 加入 或 不加入 子集，分别进行递归
递归结束条件，枚举完所有位置 i == nums.length

(2)位运算求出每个子集，用二进制枚举

用一个长度为 n 的二进制数来表示所有的子集，每一位对应nums 的每一位取还是不取（1表示取，0表示不取）
这正好能够表示 nums 数组所有的子集情况（n 个数的子集个数为 2 ^ n）
再枚举这个二进制数的每一位，如果这一位为 1 ，那么将 该位 nums 进行异或，最后将每种子集的结果累加即可

链表

环形链表

1、先要判断是否有环

判断是否有环，这个在 环形链表I 当中使用fast指针每次移动两个结点 和slow指针每次移动一个结点 当两者相等时表示有环。
(fast相对slow指针相对运动，每次接近一个结点，直到重合，所以fast不会错过slow)

2、再找出环的入口位置

环的入口位置，新建一个指针从head开始移动和slow指针一起，每次移动一格，**两者相遇结点即为环入口结点**

证明：

设链表长度为 a + b ，a为环入口距离(不包括链表入口结点)，b为环长度，f为fast指针，s为slow指针

f=2s （快指针每次2步，路程刚好2倍）
f = s + nb (相遇时，fast刚好多走了n圈)

推出相遇时slow指针走了： s = nb
从head结点走到入环点需要走： a + nb， 而slow已经走了nb，那么slow再走a步就是入环点了

所以，从head开始，和slow指针一起走，相遇时刚好就是a步

双指针

双指针并不属于某一种数据结构，只是一种操作方法，所以我们在数组、链表、字符串中都用到了。

双指针之所以被使用，是因为他可以通过两个指针在一个for循环下实现两个for循环所做的事情。这样就极大的减少了运行时间，将时间复杂度由O(n^2)降为O(n)

双指针具体怎么用就要根据需求。

比如要是反转字符串：

一个指针就要放在字符串开始，另一个放在字符串结尾，在元素交换之后，两个指针都往中间走。那这样的操作什么时候截止呢，就是要左边的指针小于右边指针，这样思路就出来了。

要是换成移除数组中的某个元素：

就要使用快慢指针，一个在前检测是不是等于要移除的元素，不等于的时候两个指针一起走就行了，但是一旦发现了要移除的元素，后面指针+1，前面那个不动，这个就可以实现覆盖了，后面的覆盖掉前面的，不就移除了吗，数组的存储是连续的，所以我们想要移除的时候就是后面元素覆盖前面的，达到移除的目的。其实替换空格也是这个道理，多了一步根据空格数来扩充空间，左右两个指针分别从原字符串和扩充字符串的最右端开始，右指针就是根据左指针给的信息（是不是空格）来行动。

反转链表就和反转字符串不一样，结构决定的：

需要把节点的指针往回指达到反转的目的，所以需要两个指针（cur，pre）指向现在这个结点，以及他之前的一个结点（因为你要往前指才能反转），注意，在往前指之前要保存cur.next，不然直接指，那cur.next就丢了，这就是链表的特殊之处，指完之后两个指针后移就可以，直到cur ==null，这就是循环的条件了。最后返回的是头指针哦，通过头指针就能找到这个链表。

移除链表元素的时候需要明白虚拟头指针的概念，这是为了统一操作，不然头指针还得单独操作，放一下卡哥的链接

移除链表元素

虚拟头结点的关键就是你先声明一个节点`ListNode dummy= new ListNode();`这个括号里面存不存值都是可以的，反正也没啥用，然后就要把这个节点指向头结点，这样你才是虚拟头结点啊`dummy.next = head;`

哈希表

哈希函数

哈希函数是一种映射关系，根据关键词key，经过一定函数关系得到元素的位置。

常见的哈希函数构造方法：

1、直接定址法：

取关键字或关键字的某个线性函数值为散列地址。

H(key) = a * key + b，其中a和b为常数

2、除留余数法：

取关键字被某个不大于散列表长度 m 的数 p 求余，得到的作为散列地址。

H(key) = key % p, p < m

3、叠加法：

将关键字分为位数相同的几部分，然后取这几部分的叠加和（舍去进位）作为散列地址。

用于关键字位数较多，并且关键字中每一位上数字分布大致均匀。

4、随机数法：

选择一个随机函数，把关键字的随机函数值作为它的哈希值。

通常当关键字的长度不等时用这种方法。

当关键字是整数类型时就可以用除留余数法；如果关键字是小数类型，选择随机数法会比较好

哈希冲突

不同关键字通过相同哈希哈数计算出相同的哈希地址，该种现象称为哈希冲突或哈希碰撞。

哈希冲突的解决：

1、开放地址法（前提是散列表的长度大于等于所要存放的元素）

发生哈希冲突后，按照某一次序找到下一个空闲的单元，把冲突的元素放入。

2、线性探查法

从发生冲突的单元开始探查，依次查看下一个单元是否为空，如果到了最后一个单元还是空，那么再从表首依次判断。如此执行直到碰到了空闲的单元或者已经探查完所有单元。

3、平方探查法

从发生冲突的单元加上12,22,32,...,n2，直到遇到空闲的单元

4、双散列函数探查法

定义两个散列函数，分别为s1和s2，s1的算法和前面一致，s2取一个1～m-1之间并和m互为素数的数。s2作为步长。

更适合于造表前无法确定表长的情况；平均查找长度较短；适合结点规模较大时。

5、分离链接法：

将哈希值相同的元素构成一个链表，head放在散列表中。一般链表长度超过了8就转为红黑树，长度少于6个就变为链表。 缺点：指针需要额外的空间

树

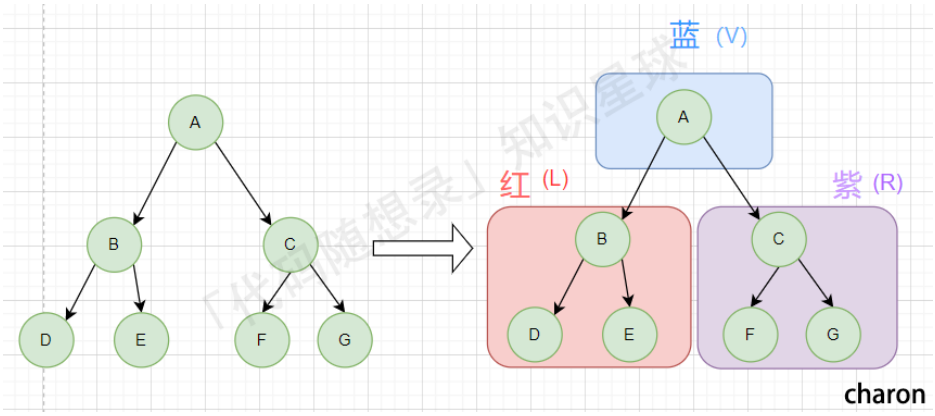
树的遍历

在学习链表和数组这两种 线性的 数据结构的时候，元素之间的次序是十分容易看出的，毕竟元素的位置就是排着队的，自带次序关系，如果想要遍历整个数据结构，无非就是选择倒着遍历还是正着遍历。

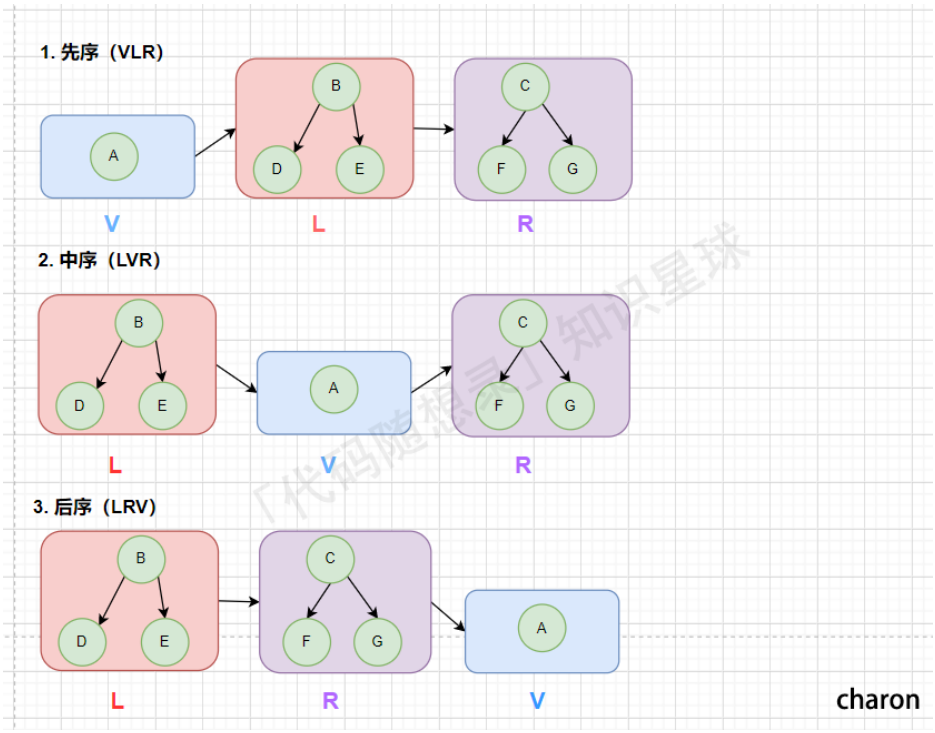
但是在学习二叉树这种半线性结构的时候，也想要遍历整棵树，可是树没有数组、链表这么明显的次序关系，所以需要人为的定义次序关系，即前中后序（VLR，LVR，LRV），这一个过程本质上就是将树的半线性结构转换成线性结构。

而对于任意一个树节点，选择前中后序中的任意一种，就等于将这个树节点和它的左右孩子的顺序给出。以前只是机械的记忆顺序。

下面举例说明，这是一颗二叉树。由整体思维可以将这个二叉树看成三个大的结点。



即蓝红紫结点，按照我们人为规定的次序，先序，中序，后序整体进行排序



这个时候**整体上**就将一棵二叉树排好序了。

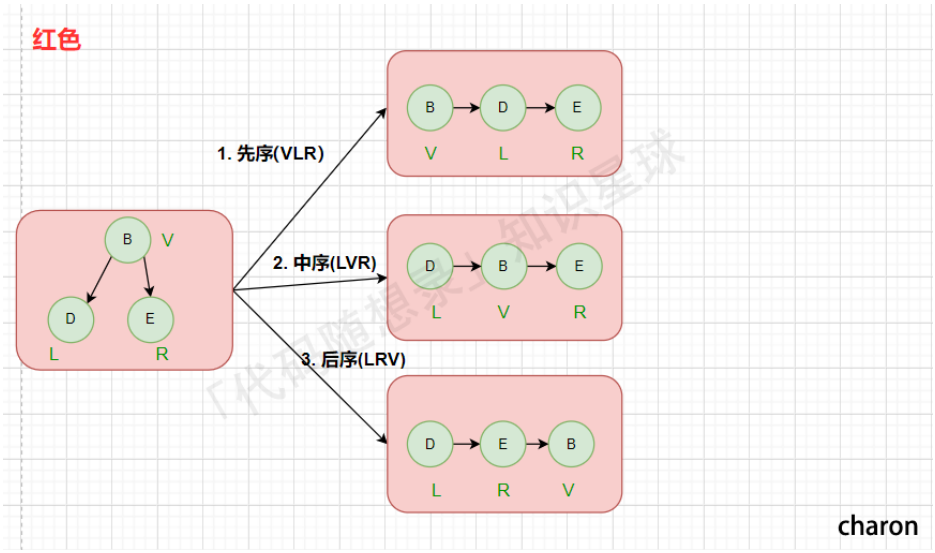
那么现在需要**局部的排序**。

局部的排序也需要按照实现人为规定的次序。

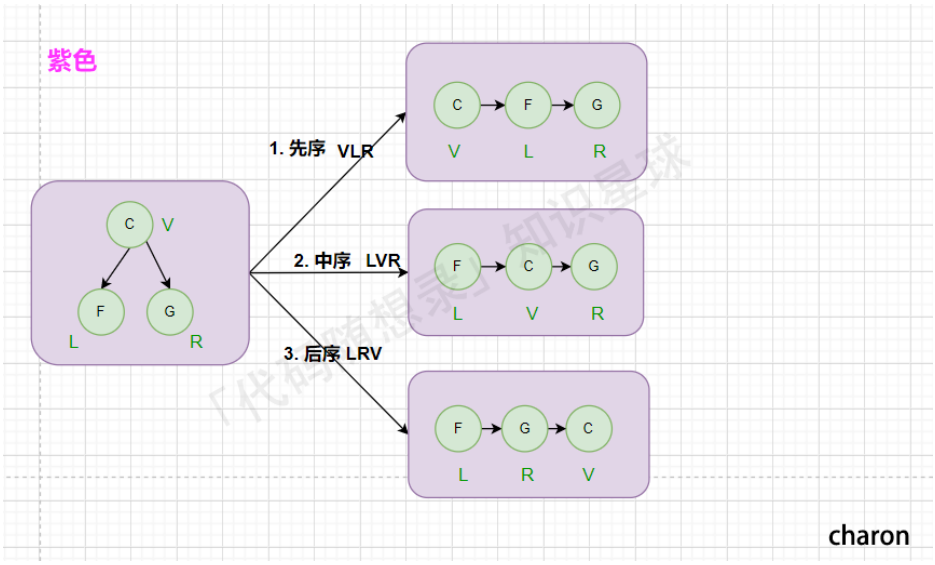
由于蓝色结点只有一个元素，所以前中后序都只有他自己一个，就不必排序

所以下面只需要看红色和紫色。

红色子树局部排序

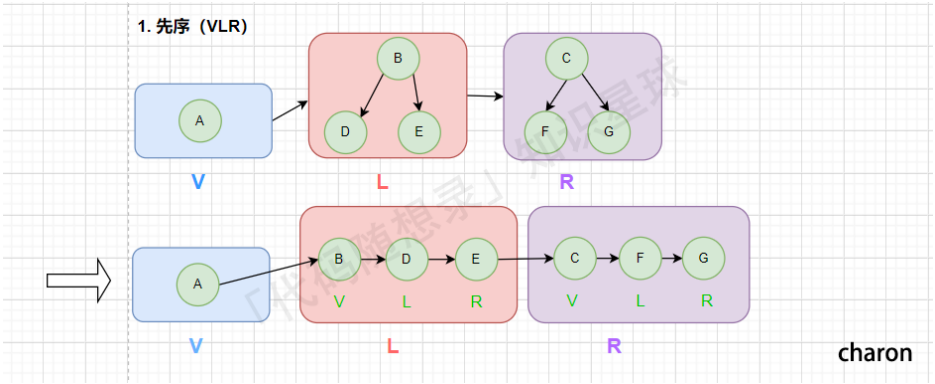


紫色子树局部排序

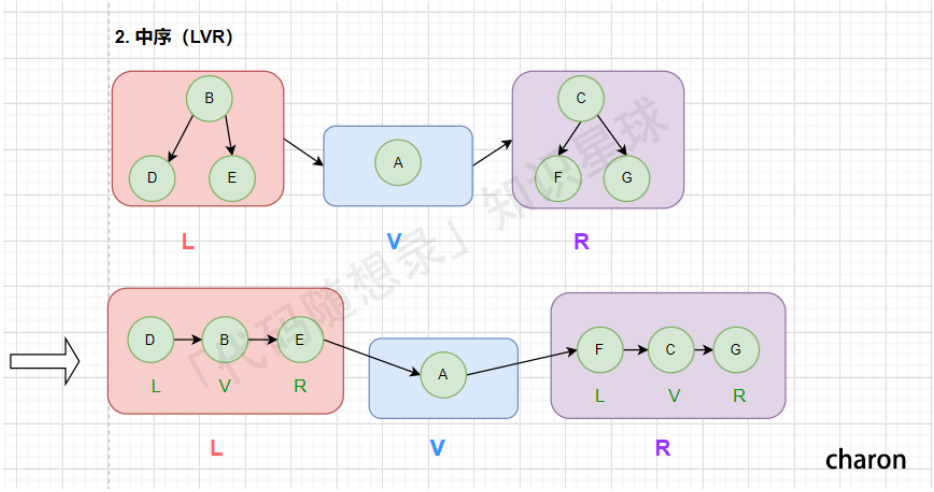


最后将局部排序结果和相同顺序的整体结果组合。

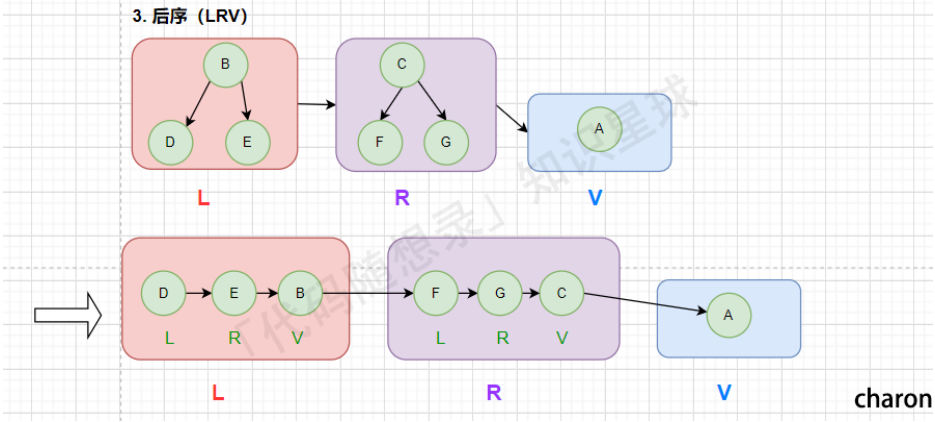
先序遍历：



中序遍历：



后序遍历：



红黑树与AVL树

红黑树

红黑树是一种二叉查找树，但在每个节点增加一个存储位表示节点的颜色，可以是红或黑（非红即黑）。

通过对任何一条从根到叶子的路径上各个节点着色的方式的限制，红黑树保证最长路径不超过最短路径的二倍，因而近似平衡。

红黑树是一种弱平衡二叉树，相对于要求严格的AVL树来说，它的旋转次数少，所以对于搜索，插入，删除操作较多的情况下，通常使用红黑树。

性质：

1. 每个节点非红即黑
2. 根节点是黑的;
3. 每个叶节点（叶节点即树尾端NULL指针或NULL节点）都是黑的;
4. 如果一个节点是红色的，则它的子节点必须是黑色的。
5. 对于任意节点而言，其到叶子点树NULL指针的每条路径都包含相同数目的黑节点;

平衡二叉树（AVL树）

红黑树是在AVL树的基础上提出来的。

平衡二叉树又称为AVL树，是一种特殊的二叉排序树。

其左右子树都是平衡二叉树，且左右子树高度之差不超过1。

红黑树较AVL树的优点

AVL 树是高度平衡的，频繁的插入和删除，会引起频繁的rebalance，导致效率下降；

红黑树不是高度平衡的，算是一种折中，插入最多两次旋转，删除最多三次旋转。

所以红黑树在查找，插入删除的性能都是O(logn)，且性能稳定，所以STL里面很多结构包括set、map底层实现都是使用的红黑树。

字典树

性质：

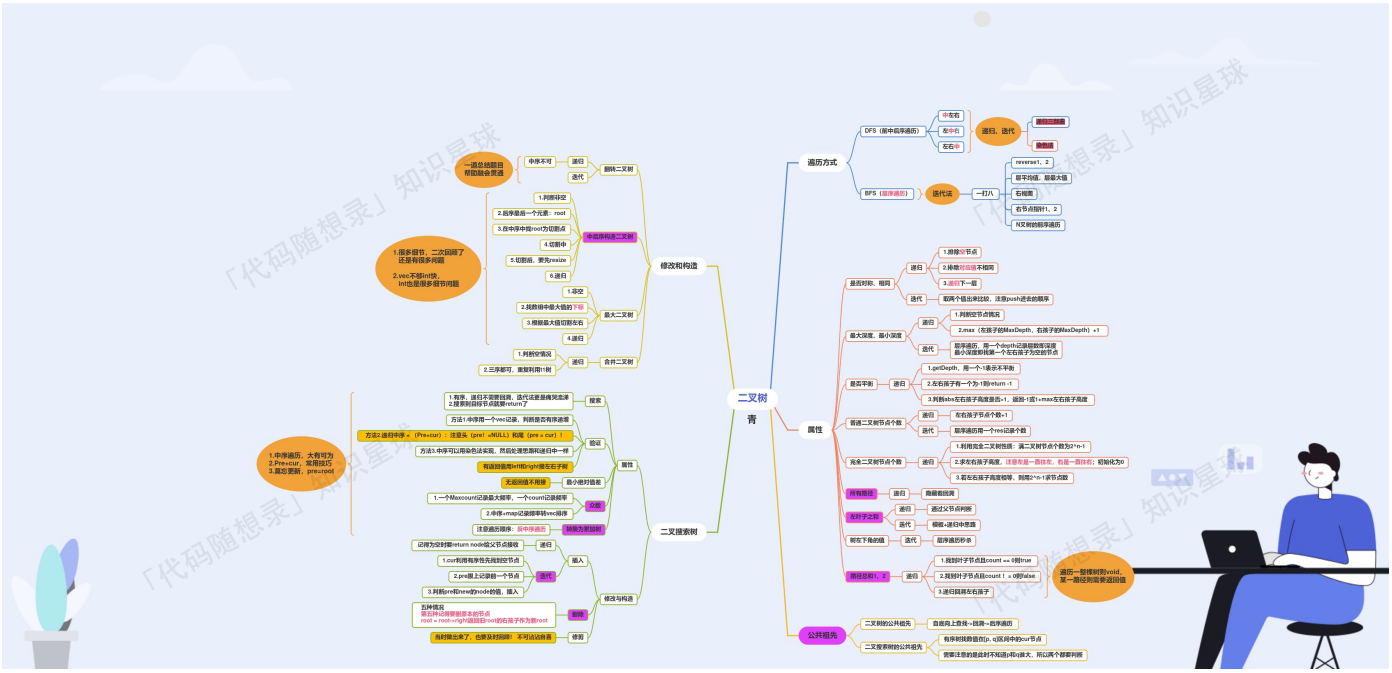
1. 根节点不保存字符，每个结点保存一个字符
2. 从根节点到某一节点。路径上经过的字符连接起来，就是该节点对应的字符串
3. 每个节点的所有子节点包含的字符都不相同

应用：统计，排序，保存大量字符串

优点：利用字符的公共前缀减少查询时间，最大限度的减少无谓的字符串比较

二叉树

原图链接：[「代码随想录」知识星球]



将有序数组转换为二叉搜索树

这个题和之前写的 t106.从中序与后序遍历序列构造二叉树 基本一致。

不一样的地方是，本题的切分点是数组nums的中间位置。然后按照以下步骤：

1. 终止条件：数组为空,return null
2. 切分点：index = left + （right-left）/ 2
3. 新建根节点：TreeNode root = new TreeNode(nums[index]);

> 如果数组nums只剩下一个元素，则return root

4. 分割数组：[0, index) [index + 1, len)
5. 递归左右遍历

> 注意【不变量】，后来[左闭右开) 和[左闭右闭]都过了，还是觉得这个题[左闭右闭]友好一些

把二叉搜索树转换为累加树

直接 【右/根/左】反中序遍历一遍二叉树进行了。

1. 由于遍历整颗树，而且不用增/删BST，所以递归不需要返回类型。
2. 用一个sum 记录当前节点之前的节点和 sum
3. 右 -> 递归遍历右子树
4. 根 -> 逻辑处理
5. 左 -> 递归遍历左子树

> 这个题也可用迭代法，要用栈。仍然是反中序【右/根/左】

回溯

1. 组合问题

1. 对于同一集合问题：需要使用startIndex记录横向遍历的起始位置
2. 给定数组中的元素是否可以重复多次选择问题：若可重复选择，下一层递归的起始位置为 i；若不可重复选择，下一层递归的起始位置 i

2. 组合总和（组合问题 +求和）

剪枝操作（关键）：

```
if(sum > target) return;
// 1. 当前累加和已经大于目标和
sum + candidates[i] <= tartget;
// 2. 先判断当前和是否大于目标和，是则不累加下一层的节点
```

3. 分割（组合问题）

实质也是组合问题，切割的方式就是选择的起始位置

关键是截取子串是什么：`s.substr(startIndex,i-startIndex+1);`

4. 排序问题

需要使用标记数组，记录当前元素是否访问过，访问过设置为true：

`vector<bool> used(candidates.size(),false);`

组合排序通用问题、给定vector中可能包含重复的元素，要求不能出现重复的组合（即为树的树枝的节点可以重复，但是同一层不允许重复）

- 解决方法：
  前提：需要先对数组升序【排序】，保证相同元素相邻，便于判断前后是否相等【关键】
  1. 定义标识数组 vector used(candidates.size(),false);
  2. 标识数组的含义：
     - `used[i - 1] ==true`：表示同一树枝使用过;
     - `used[i - 1] == false`，说明同一树层`candidates[i - 1]`使用过;
     - 做选择时 `used[i] =true` 撤销选择时：`used[i] = false`;
  3. 判断一定是前一个元素，因为要和当前元素比较;
  4. 真正的去重代码：if(i>0&&candidates[i-1] == candidates[i] && used[i-1] ==false) continue;

贪心

构成交替字符串需要的最小交换次数

题目要求最小交换次数，那么对于不同的串，0 1 的出现次数也不相同，根据字符串的长度可分为偶数和奇数
先求出 num1， num0，odd1，odd0
首先特判，num1 和 num0 两者个数相差不能大于 1 ，否则返回 -1

对于偶数长度，有两种情况，1010...，0101...
1 开头，那么奇数位置都应该为 1 ，那么奇数位置为 0 的次数(odd0) 就是要交换的次数
0 开头，那么奇数位置都应该为 0 ，那么奇数位置为 1 的次数(odd1) 就是要交换的次数
要求最小交换次数，返回两者中的最小值

对于奇数长度，有两种情况，010... ，101....
由于是只有奇数个，所以 1 开头的话，必然是 num1 > num0，要交换的次数也就是odd0，
反之，0开头，必然num0 > num1，返回odd1

动态规划

方法论

动态规划中每一个状态一定是由上一个状态推导出来的，这一点就区分于贪心，贪心没有状态推导，而是从局部直接选最优的。

dp五步曲

1. 确定dp数组（dp table）以及下标的含义
2. 确定递推公式
3. dp数组如何初始化
4. 确定遍历顺序
5. 举例推导dp数组

注意：

- dp 初始化 case 的情况，一般要放在函数开头前单独处理，防止dp数组index越界....
    - 比如，509. 斐波那契数 if (n <= 1) return n; 需要单独处理。
    - 否则，当n==1时，初始化访问dp[1]数组将越界...

dp debug方法

打印 dp 数组日志，并查看与自己想模拟的dp值是否一致

> 若 dp 值与自己模拟的结果一致，说明是 dp五步曲中的2、3、4步骤出现问题；
> 若 dp 值与自己模拟的结果不一致，说明是自己代码细节有问题

做 dp 题目之前，一定要把状态转移在 dp 数组的上模拟一遍，心中有数，确定最后推出的是想要的结果。

初识DP

509. 斐波那契数

题目描述

> 返回第 n 个斐波那契数

按照 dp 五部曲来

1. 确定dp数组（dp table）以及下标的含义
2. int[] dp = new int[n + 1];
3. dp[i] 表示第 i 个 fib
4. 确定状态转移方程：dp[i] = dp[i- 1] + dp[i - 2]
5. 初始化：dp[0] = 0; dp[1] = 1;
6. 确定遍历顺序：从左到右
7. dp长度为9时 dp元素为：0、1、1、2、3、5、8、13、21

70. 爬楼梯

题目描述

> 假设你正在爬楼梯。需要 n 阶你才能到达楼顶。
> 每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢?

和 509.斐波那契数 很类似，当前台阶的步数可以由于前两个台阶之和决定。

即，当前台阶由前一步台阶跳一步而来 或者 由前两步台阶一次跳两步而来。

为什么不能由前两步台阶分别跳两个一步到当前台阶而来呢?

因为前两步台阶跳一步之后就到了当前台阶的前一步台阶，仍然相当于是从当前台阶的前一步台阶跳一步而来的。。。

746. 使用最小花费爬楼梯

dp 五部曲

1. dp[i] 表示上第 i 层楼梯最小花费
2. 状态转移方程 : dp[i] = Math.min(dp[i - 1] + cost[i - 1], dp[i - 2] + cost[i - 2]);
3. 初始化 ： dp[0] = dp[1] = 0;
4. 顺序：从左向右
5. 打印日志
6. 如果cost 为 {1, 100, 1, 1, 1, 100, 1, 1, 100, 1}
7. 对应 dp 应为 {0, 0, 1, 2, 2, 3, 3, 4, 4, 5}

注意：

这里不能直接 return dp[len - 1]，顶峰为最后一个元素后面，但是这里的 dp[n-1] 还没到达顶峰.

顶峰需要从 dp[n-1]再走一步，或者从dp[n-2]一次走两步

应该 return Math.min(dp[len - 1] + cost[len - 1], cost[len - 2] + dp[len - 2]);

62. 不同路径

题目描述:

> 一个机器人位于一个 m x n 网格的左上角 （起始点在下图中标记为 “Start” ）。
>
> 机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 “Finish” ）。
>
> 问总共有多少条不同的路径?

dp 五步曲：

1. 定义dp数组、确定dp[i][j]含义
2. int dp = new intm;
3. dp[i][j]表示到达(i,j)路径的数量
4. 递推公式：dp[i][j]= dp[i][j]- 1 + dp[i][j];
5. 当前位置只能由其正上方和正左方过来
6. 初始化
7. 第一行、第一列全为 1
8. if (i == 0 || j == 0) dp[i][j]= 1;
9. 顺序：从上到下，从左到右
10. 打印dp日志

63. 不同路径 II

题目描述：

> 一个机器人位于一个 m x n 网格的左上角 （起始点在下图中标记为“Start” ）。
>
> 机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。
>
> 现在考虑网格中有障碍物。那么从左上角到右下角将会有多少条不同的路径

这题和上题 62. 不同路径 基本类似

区别在于两点：

1. 碰到障碍物时：dp[i][j] = 0
2. 初始化不同：如果第一行中间有障碍物了，则其后均不可达（其后dp0=0）；第一列，同理
3. 这里 carl哥 初始化的相当巧妙
4. 循环终止条件，一旦遇到obstacleGridi == 1的情况就停止dpi的赋值1的操作
5. for (int i = 0; i < m && obstacleGrid[i][i] == 0; i++) dp[i][j] = 1;

343. 整数拆分

题目描述：

> 给定一个正整数 n，将其拆分为至少两个正整数的和，并使这些整数的乘积最大化。 返回你可以获得的最大乘积。

dp 五步曲：

1. 定义dp数组、确定dp[i]含义
    1. int[] dp = new int[n + 1];
    2. dp[i] 表示将i拆分后的最大乘积

2. 递推公式：dp[i] = Math.max(dp[i], Math.max(j *(i - j), j* dp[i - j]));
    1. 获取dp[i]的两种方式
        1. j * (i - j) 直接相乘
        2. j * dp[i - j] 相乘
    2. 如果上述两种所得结果小于当前 dp[i]，则不更新
    3. 初始化：dp[2] = 1;
        1. dp[0]、dp[1]没意义
    4. 顺序：从左到右
    5. 打印dp

96. 不同的二叉搜索树

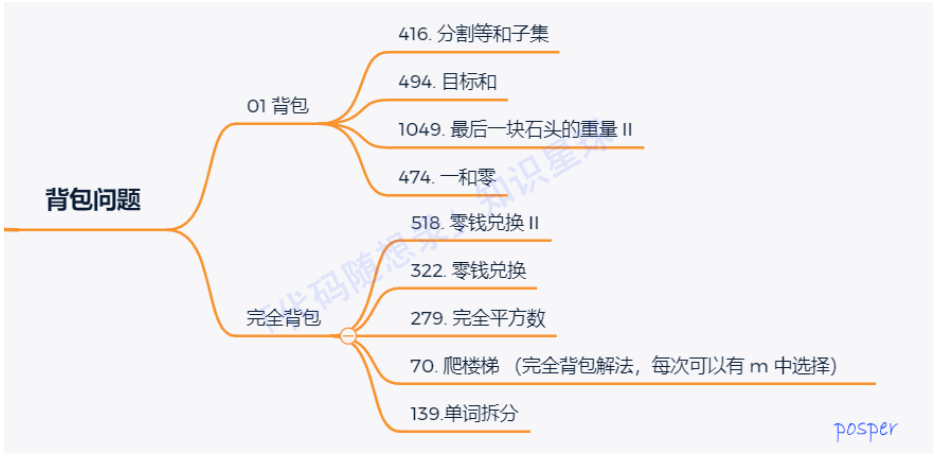
题目描述

> 给你一个整数 n ， 求恰由 n 个节点组成且节点值从 1 到 n 互不相同的 二叉搜索树 有多少种？返回满足题意的二叉搜索树的种数。

dp 五步曲

1. dp[i] 表示从1到i的BST种数
2. 状态转移方程：dp[i] += dp[j - 1] * dp[i - j];
3. 初始化
    1. dp[0] = 1;
    2. dp[1] = 1;
    3. dp[2] = 2;
4. 顺序：从左到右
5. 打印dp日志

背包问题



322. 零钱兑换

题目描述：

> 给定不同面额的硬币 coins 和一个总金额 amount。编写一个函数来计算可以凑成总金额所需的最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1。
>
> 你可以认为每种硬币的数量是无限的。

和 518.零钱兑换II 区别在于本题返回凑成总金额所需的最少的硬币个数，而 518.零钱兑换II 返回可以凑成总金额的硬币组合数

区别在于：

1. 递推公式不同
    - 本题：dp[j] = Math.min(dp[j], dp[j - coins[i]] + 1);
    - 518.零钱兑换II：dp[j] += dp[j - coins[i]]; // 求总和
2. 初始化不同
    - 本题 dp[0] = 0; 然后其他元素初始值设为 正无穷
    - 518.零钱兑换II 中 dp[0] = 1;
3. 遍历顺序要求不同
    - 本题：求钱币最小个数，那么钱币有顺序和没有顺序都可以，都不影响钱币的最小个数。
    - 518.零钱兑换II求得是组合数，一定要外层for循环遍历物品，内层for遍历背包。

> 如果dp[j - coins[i]]是初始值则跳过，if (dp[j - coins[i]] == INT_MAX) continue;

dp五步曲：

1. dp[i] 表示总金额为 i 所需的最少的硬币个数
2. 递推公式：dp[j] = Math.min(dp[j], dp[j - coins[i]] + 1);
3. 初始化
    - dp[0]=0;
    - 其他的初始化为正无穷
4. 遍历顺序
    - 此时，内外层顺序无关
    - 完全背包物品可以重复使用，所以内层正序（01背包中一维dp，内层必须倒序，保证只用一次）
5. 打印dp

> 如果dp[j - coins[i]]是初始值则跳过, if (dp[j - coins[i]] == INT_MAX) continue;

279. 完全平方数(完全背包)

题目描述：

> 给定正整数 n，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于 n。你需要让组成和的完全平方数的个数最少。
> 给你一个整数 n ，返回和为 n 的完全平方数的 最少数量 。
> 完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

这个和上面一题 322.零钱兑换 基本一致

不同在于：

1. 本题递推公式：dp[j] = Math.min(dp[j], dp[j - i * i] + 1);
2. 满足条件才更新dp公式：if (j >= i * i && dp[j - i * i] != Integer.MAX_VALUE)
    - 本题内外层顺序无关
    - 这里我写的是 外层物品，内层背包（但是，这个题内外反过来也可以）

dp五步曲：

1. dp[i] 表示和为 i 的完全平方数的最小数量
2. 递推公式：dp[j] = Math.min(dp[j], dp[j - i * i] + 1);
3. 初始化 （同 322. 零钱兑换）
    - dp[0]=0;
    - 其他的初始化为正无穷
4. 遍历顺序
    - 本题内外层顺序无关，这里我写的是 外层物品，内层背包
    - for (int i = 1; i < dp.length; i++) // 外层物品，内层背包
        - for (int j = i * i; j < dp.length; j++) // 内层背包
5. 打印dp

爬楼梯

这个题之前那样写的只能处理一次上一层、两层，但是如果一次能上1，2，...或 m 层呢?

> 如果是每次只能上小于m的奇数步的话，只用采取一个数组 steps 存放每次可以上楼梯的方案就行

那之前的就不太好写了，这时候其实这是个完全背包问题，因为：

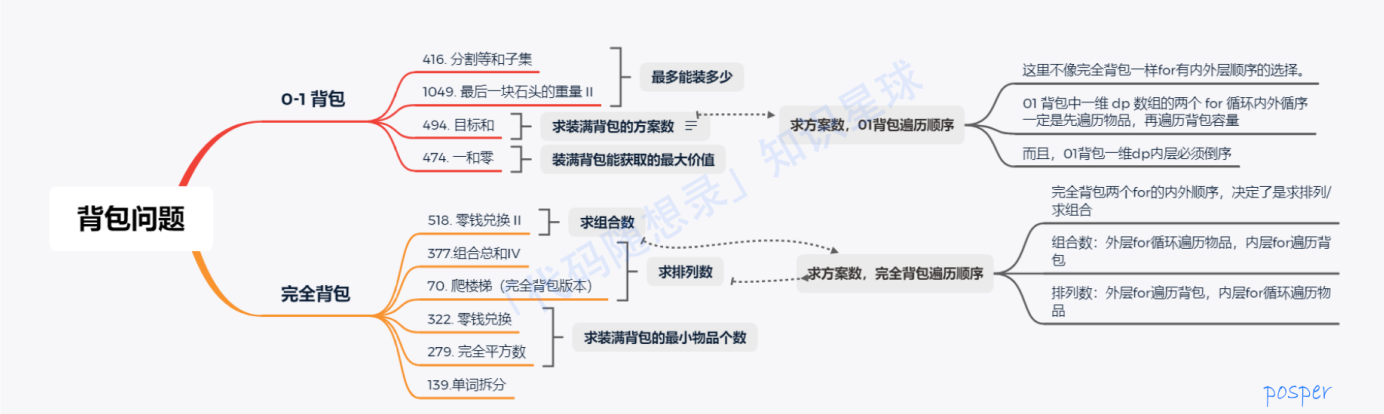
1. 每次上的楼梯数（1阶，..m阶）看成物品，楼顶台阶数就是背包容量
2. 每个台阶数（相当于物品）可以重复使用，所以是完全背包
3. 然后，求得是能上到楼顶（相当于吧背包装满）的方案数量
    1. 好了好了，背包装满的方案数量，就有如此相像。那不就是组合数或者排列数嘛！
    2. 这个肯定是排列数，因为(1,2)和（2,1）是两种不同的方法

既然转化成求完全背包的排列数问题，那么就是套模板了

dp五步曲：

1. dp[i] 表示上到第 i 层的方案树
2. 完全背包（排列数固定递推公式）： dp[j] += dp[j - i];
3. 初始化：dp[0] = 1;
    - 排列数固定初始化。又因为方案均为正数，其他的初始化为0就行了
4. 遍历顺序（排列数：必须是外层背包，内层物品）
5. 打印 dp

背包小总结 (01背包、完全背包)



背包问题最重要的就是两点：递推公式和遍历顺序

背包问题面试常考的就是 01背包和 完全背包。

其中 01背包中物品只能选一次，完全背包中物品可以无限次选取

背包类型主要有两种题型：

1. 求使背包装尽可能装满能获取的最大/最小价值 或 装满背包的最大/最小物品个数
    - 递推公式
        - 最大/最小价值：dp[j] = max or min(dp[j], dp[j - weight[i]] + value[i]);
        - 最大/最小物品个数：dp[j] = Math.max or min(dp[j], dp[j - coins[i]] + 1);
    - 初始化
        - 最少：dp[0]=0; 其他的初始化为正无穷
        - 最多：全初始化为0就行（前提是coins中元素全为正数；否则，需要把负元素在dp中对应的位置初始化为负无穷）
2. 求使背包装满的方案数（主要有两种情况 组合数（顺序无关）和 排列数（顺序有关））
    - 递推公式：都是 dp[j] += dp[j - nums[i]];
    - 但是二者内外层的遍历顺序不同：
        - 组合数：外层for循环遍历物品，内层for遍历背包。
        - 排列数：外层for遍历背包，内层for循环遍历物品。

背包的遍历顺序：

1. 01 背包
    - 01 背包中二维 dp 数组的两个 for 遍历的内外循序是可以颠倒的
        - 这里内外层都是从小到大遍历
    - 01 背包中一维 dp 数组的两个 for 循环内外循序一定是先遍历物品，再遍历背包容量
        - 注意：内层循环（背包容量）必须倒序遍历，保证每个物品只取一次。
2. 完全背包
    - 在完全背包中，对于一维 dp 数组来说，其实两个 for 循环嵌套顺序同样无所谓（仅限于纯完全背包问题！）

注意：完全背包求 组合数/排列数 时，两个 for 的内外顺序有关！！！
求最大/最小价值（纯完全背包问题）时，这时完全背包两个 for 内外顺序无关！

例：

		01背包问题
遍历顺序		先正序遍历物品，后**逆序**遍历容量
题型	装满背包，求最大价值 416.分割等子集 1049.最后一块石头的重量	1．dp的初始化：vector<int> dp(bag + 1,0); 2．数量：一般是包的容量 + 1；便于取到dp[bag] 3．值：全部设置为0 4．递推式：**dp[j] = max(dp[j], dp[j - weight[i] + value[i])** 5．拓展：最后多了一步判断：是否能达到最大价值 【416】；或者均分成两份之后，剩余多的部分【1049.】
	装满背包的组合数 494.目标和	1．dp的初始化：vector<int> dp(bag + 1,0); 2．数量：一般是包的容量 + 1；便于取到dp[bag] 3．值：**dp[0] = 1**；便于递推；其他的全部初始化为0 4．递推式：**dp[j] += dp[j - nums[i]]** 5．拓展：关键是找到包的容量！！！
	装满背包的**最多**物品数目 474.一和零	1．dp的初始化：vector<int> dp(bag + 1,0); 2．数量：一般是包的容量 + 1；便于取到dp[bag] 3．值：全部设置为0 4．递推式： **dp[j] = max(dp[j],dp[j - nums[i]] + 1)**【一维包的容量】 dp[i][j] = max(dp[i][j],dp[i - num1][j - num2] + 1)【二维包的容量】

题型	遍历顺序	完全背包问题
组合数 518 零钱兑换 II	先正序遍历**物品**，后正序遍历**背包**	1．dp的初始化：vector<int> dp(bag + 1,0); 2．数量：一般是包的容量 + 1；便于取到dp[bag] 3．值：dp[0] = 1;其余的为0 4．递推式：**dp[j] += dp[j - nums[i]]** 5．**细节：两个数相加，同样考虑是否越界：dp[j] < INT_MAX - dp[j - nums[i]]**
排列数 70 爬楼梯 377 组合总和 IV	先正序遍历**背包**，后正序遍历**物品**	
求最少物品数目 279 完全平方数 322 零钱兑换	组合数和排列数的两种遍历方式均可	1．dp的初始化：vector<int> dp(bag + 1,INT_MAX); 2．数量：一般是包的容量 + 1；便于取到dp[bag] 3．值：dp[0] = 0;其余的为INT_MAX，避免覆盖最小值 4．递推式：**dp[j] = min(dp[j],dp[j - nums[i]] +1)** 5．**细节：初始化赋值了最大值，所以在求解递推式时，注意考虑越界的情况：if(dp[j -nums[i]] != INT_MAX)才能使用递推式**
包是否装满 **193.单词拆分***** **难**	均可；但是选择先遍历背包，后遍历物品【不需要额外花费空间存储】	dp[i]：表示长度为i的字符串是否能由字典中的单词组成；是bool型 初始化：dp[0] = true【便于递推】；其他的全部初始化为false 递推式：dp[j] = true && [j,i]区间的字符串在字典中，可以推出dp[i]为true

子序列问题

674. 最长连续递增序列

题目描述：

> 给定一个未经排序的整数数组，找到最长且 连续递增的子序列，并返回该序列的长度。

和 300. 最长递增子序列 区别在于，本题要求递增子序列需要是连续的。

这样，这题反而会更简单一些...

1. 本题一层 for 比较 nums[i] > nums[i - 1] 就行了；
2. 不用像 300. 最长递增子序列 两层 for 比较 nums[i] > nums[j] （j在 [0, i]之间）

dp五步曲：

1. dp[i] 表示以 i 结尾（不一定要从0开始）的连续递增子序列的长度
2. 递推公式
    1. if (nums[i] > nums[i - 1]) // 确保连续 & 递增
    2. dp[i] = dp[i - 1] + 1; // 2、递推公式
3. 初始化
    1. 最小长度为 1，所有 dp[i] 初始化均为 1
4. 顺序：前到后
5. 打印dp

注意：这里不能返回 dp[n -1]，而是 return dp 数组中的 max 值

392. 判断子序列

题目描述：

> 给定字符串 s 和 t ，判断 s 是否为 t 的子序列。
> 字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，"ace"是"abcde"的一个子序列，而"aec"不是）。

这里的子序列，不要求连续。只要相对顺序一致就 ok。

dp五部曲：

1. dpi 表示s中下标[0,i-1]与t下标在[0,j-1]中，相同子序列长度为dpi
2. 递推公式：（两种情况）
    - if (s.charAt(i - 1) == t.charAt(j - 1))
        - dpi = dpi - 1 + 1;
        - 当前元素匹配，相当于当前长度为去掉 s[i-1]和t[j-1]的长度加一，即dpi = dpi - 1 + 1;
    - if (s.charAt(i - 1) != t.charAt(j - 1)) // 需要删去 t[j]
        - dpi = dpi;
        - 相当于看s[i-1]与 t[i-2]比较
3. 初始化
    - dp0 = 0; 表示空串s与t下标在[0,j-1]中，不可能有相同子序列，所以全为0
    - dpi = 0; 表示空串t与s下标在[0,i-1]中，不可能有相同子序列，所以全为0
4. 顺序：从左到右

5. 打印dp

最终，判断 return dpm == m ? true : false;

115. 不同的子序列

题目描述：

> 给定一个字符串 s 和一个字符串 t ，计算在 s 的子序列中 t 出现的个数。

dp五部曲：

1. dpi 表示 s下标在[0,i-1]中与 t下标在[0,j-1]中不同子序列的个数
    - int dp = new intm + 1;
2. 递推公式：（两类情况）
    - if (s.charAt(i - 1) == t.charAt(j - 1)) // 匹配
        - 这第一类又分为两小种情况：
            - 由于不用连续，所以，计算当前位置匹配，仍然可以选择不使用当前元素；即，dpi - 1;
            - 如果使用当前元素进行匹配，则 dpi - 1
        - 第一大类情况是这两小种情况之和
            - 即，dpi = dpi - 1 + dpi - 1;
    - if (s.charAt(i - 1) == t.charAt(j - 1)) // 不匹配，则删除s[i-1]
        - dpi = dpi - 1;
3. 初始化
    - dpi 表示删除s[i]后是空串的个数，所以全是1
    - dp0 表示删除空串s后是t[j-1]的个数，不可能，所以全是0
4. 顺序：从左到右
5. 打印dp

516. 最长回文子序列

题目描述：

> 给定一个字符串 s ，找到其中最长的回文子序列，并返回该序列的长度。可以假设 s 的最大长度为 1000 。

子串连续，但是子序列可以不连续

dp 五步曲：

1. dp[i] 表示：如果子串[i,j]范围内最长回文串的长度
    - int dp = new int[n];
2. 递推公式
    - if (s.charAt(i) == s.charAt(j)) { // 首尾匹配
        - dp[i] = dp[i] + 1 + 2;
    - if (s.charAt(i) != s.charAt(j)) { // 首尾不匹配
        - 首尾字符不匹配，则 [i,j]子串一定不是回文串，则需要看[i+1,j]和[i,j-1]中的max回文串
        - dp[i] = Math.max(dp[i] + 1, dp[i]);
3. 初始化

- 递推公式中子串 [i, i] 没有计算
- dp[i] = 1; // 一个字符铁定是回文串
- 其他初始化为 0

4. 顺序：由递推公式，dp[i]需要从其左下方推导过来。所以，要从下到上，从左到右
5. 打印 dp

总结

动态规划之子序列问题			
题型		**初始化**	**dp状态以及递推式**

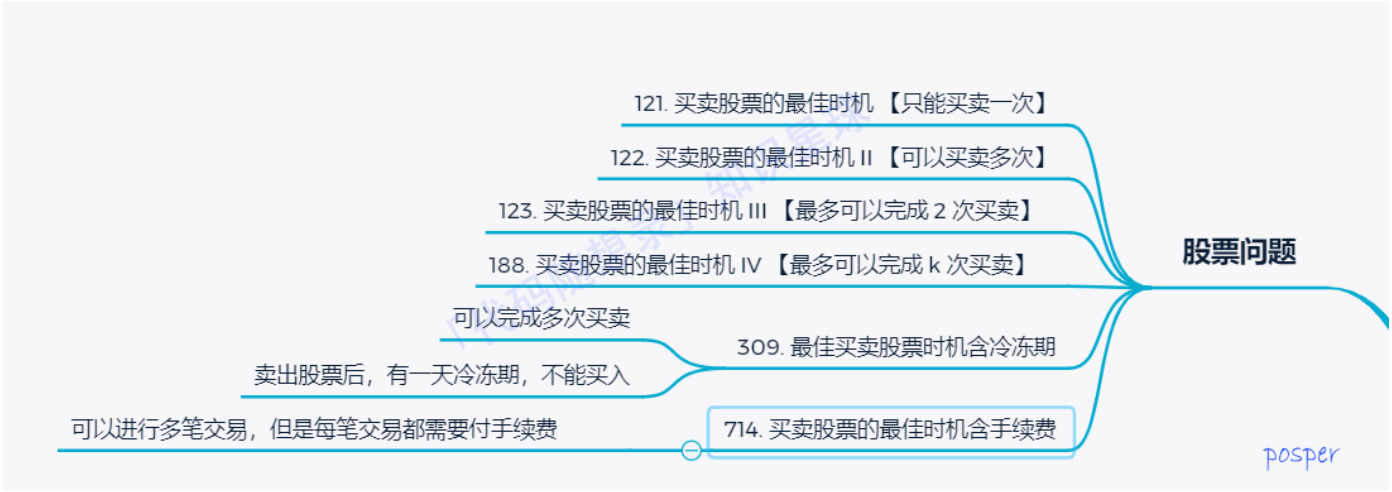
不连续子序列	300.最长递增子序【一维滚动数组】	vector<int> dp(nums.size(),1);
		均初始化为1，因为至少长度是本身
			dp[i]:以i结尾的子序列的严格递增的长度
			if(nums[j] < nums[i]) dp[i] = max(dp[i],dp[j] + 1);//需要从头开始遍历；记录中间所有的递增子序列，用到了滚动数组
			关于最大值取处：需要遍历dp数组一遍找最大dp

	1143.最长公共子序列【二维不连续问题】	vector<vector<int>> dp(len1 + 1,vector<int>(len2 +1,0)); **均初始化为0**，因为空串与其他串的公共长度0	dp[i][j]：以i-1结尾的text1和以j-1结尾的text2公共子序列的长度 text1[i-1] == text2[j-1]：dp[i][j] = dp[i-1][j-1] + 1 不相等：dp[i][j] = max(dp[i-1][j],dp[i][j-1]) **最长公共子序列长度**：dp[len1][len2]
	1035.不想交的线		

连续子序列	674.最长连续递增[严格递增]序列	vector<int> dp(nums.size(),1);
		均初始化为1，因为至少长度是本身
			dp[i]:以i结尾包括i最长连续严格递增子序列长度
			if(nums[i-1] < nums[i]) dp[i] = max(dp[i-1] + 1,dp[i]);//由于是连续；只需要比较相邻元素即可，要么取之前的，要么不取；直接去当前元素
			最长连续子序列：**设置一个变量【res = 1】记录最大长度**

	53.最大子序和	dp[0] = nums[0];	**dp[i] = max(dp[i-1] + nums[i],nums[i]);** 最大和：设置一个变脸记录最大和；res = dp[0] 因为遍历是从1开始，故需要考虑只有一个元素的情况【细节】

股票问题



技巧总结

买股票的最佳时机

注:使用二维数组 dp[][]

	限制条件	dp 数组的状态(关注 j)	初始化过程
I	买卖一次	1. 持有 dp[i][0] 2. 不持有 dp[i][1]	dp[0][0]= -prices[0] dp[0][1]= 0
II	买卖无数次		
III	买卖 2 次	1. 从未买入 dp[i][0] 2. 第一次买入 dp[i][1] 3. 第一次卖出 dp[i][2] 4. 第二次买入 dp[i][3] 5. 第二次卖出 dp[i][4]	dp[0][0]=0 dp[0][1]=-prices[0] dp[0][3]=-prices[0] dp[0][3]初始化为-price[0]的原因:当天买入第二次,可看做当天第一次买入花费 price[0]后,当天卖出又赚了 price[0],所以第二次买入花费为 price[0]
IV	买卖 k 次	1. 从未买入 dp[i][0] 2. 第 j 次买入 dp[i][2t-1] 3. 第 j 次卖出 dp[i][2t] (t 为整数)	for(int j=1;j<=2*k;j+=2) dp[0][j]=-prices[0];

含冷冻期	买卖无数次&&冷冻期	1. 持有 dp[0][0]
		2. 不持有&&当天卖出 dp[0][1]
		3. 不持有&&当天不卖出 dp[0][2]
			dp[0][0]=-prices[0];
			dp[0][1]=0;
			dp[0][2]=0;

含手续费	买卖无数次&&手续费	1. 持有 dp[i][0]
		2. 不持有 dp[i][1]
			dp[0][0]=-fee-prices[0];
			//fee 为手续费
			dp[0][1]=0;

714. 买卖股票的最佳时机含手续费

可以进行多笔交易，但是每笔交易都需要付手续费

和 122. 买卖股票的最佳时机 II 很像

唯一区别在于递推公式中在卖出操作时，本题需要额外减去手续费

注意：

1. 每笔交易你只需要为支付一次手续费
2. 本题选择在卖出股票时支付手续费

dp 五步曲：

1. 确定 dp 及其含义
    1. 每天只有 2 个状态
        1. dp[i]: 第 i 天持有股票获得的最大现金
        2. dp[i]: 第 i 天卖出股票获得的最大现金
    2. `int[] dp = new int[n];`
2. 递推公式
    1. `dpi = Math.max(dpi - 1, dpi - 1 - prices[i]);`
        1. 今天不买入，则保持前一天买入状态，为 dp[i] - 1
        2. 今天买入，则需要在前一天卖出的基础上减去今天的股票价格，为 dp[i] - 1 - prices[i]
    2. 二者，取 max
    3. dp[i] = Math.max(dp[i] - 1, dp[i] - 1 + prices[i] - fee);
        1. 这里要多剪去一个 手续费
        2. 今天不卖出，则保持前一天卖出状态，为 dp[i] - 1
        3. 今天卖出，则需要在前一天买入的基础上加上今天的股票价格，再减去手续费
    4. 取 max
3. 初始化
    1. `dp[0] = -prices[0];`
    2. 其他为 0
4. 顺序：从前到后
5. 打印 dp

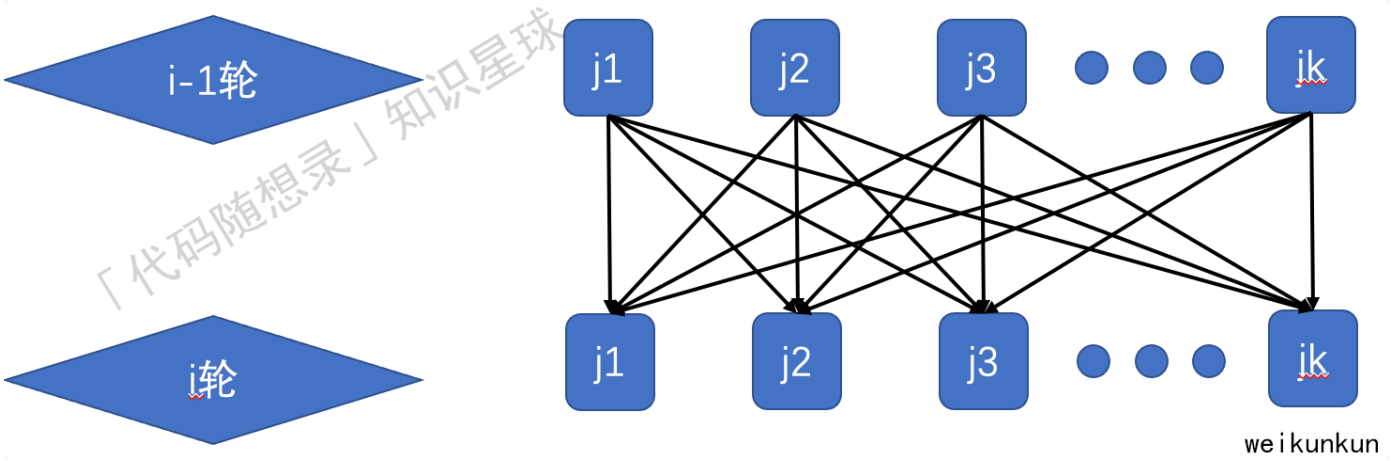
时间序列模型

时间序列模型(给出一个序列(数组/字符串)，其中每一个元素可以认为“一天”，并且“今天”的状态只 取决于“昨天”的状态（无后效性？）)

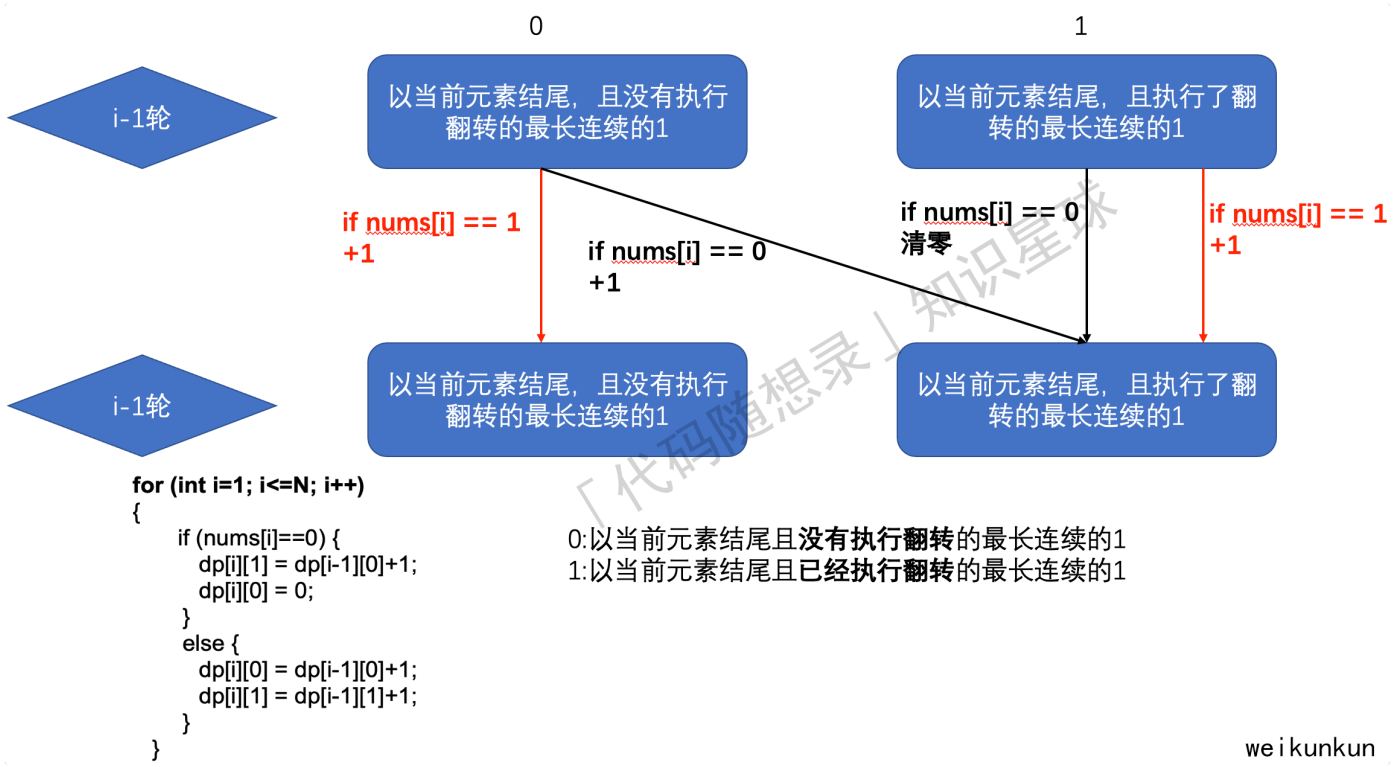
后无效性：

思路：

1. 确定dp数组dp[i][j]（第ith轮的第j种状态）；
2. 画图，千方百计的将dp[i][j]和前一轮的状态dp[i-1][j]产生关系（确定状态转移方程）；
3. 确定基础值，最后开始硬编（感觉这种模型的dp真套路，不过有时候无法通过题目定位出是这种类型的题目）



举例说明：LC_487. 最大连续1的个数 II



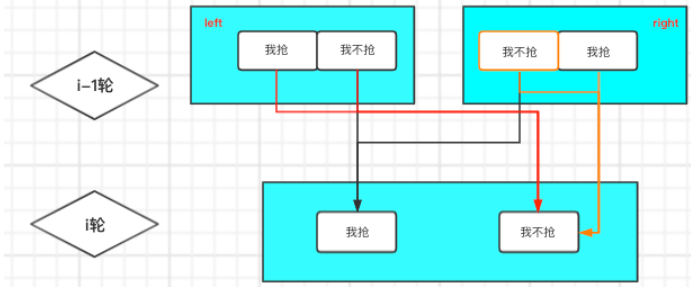
代表性的题目： 打家劫舍系列、股票系列，打家劫舍III运用了树形DP。

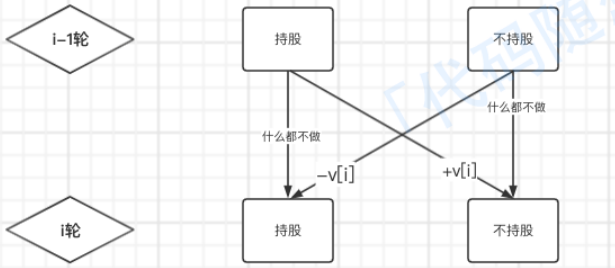
时间序列模型（状态机模型），这种类型的题目，在确定dp的数组和下标的含义之后，状态的转换可以通过话状态图得到

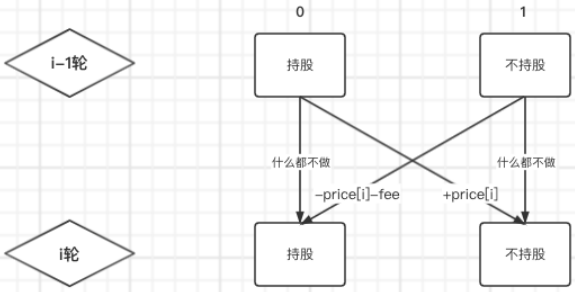
之后的初始化等操作也就自然就出来了。

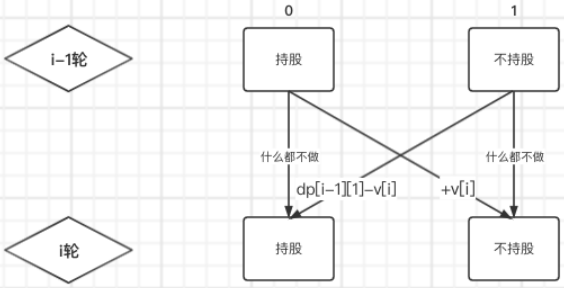
或者说是后序遍历的深入理解吧，需要明确当前层构成的结果如何计算，以及需要向上传递什么结果。可以先从二叉树的路径问题入手理解。

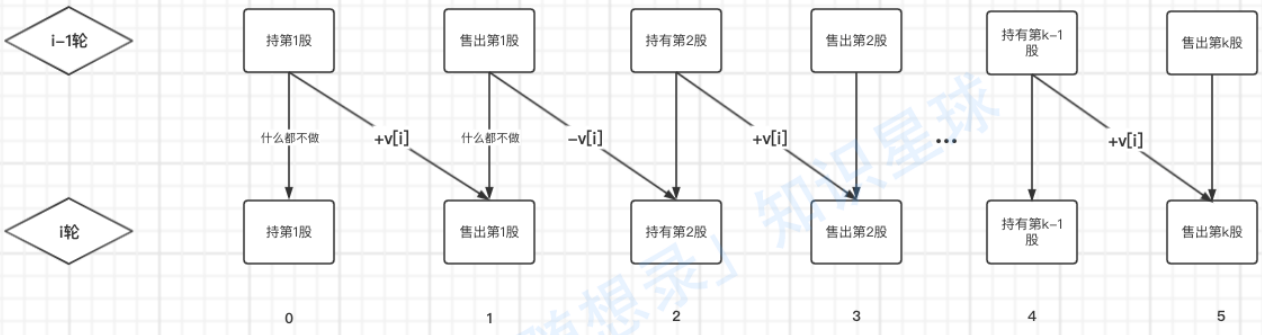
原图链接：「代码随想录」知识星球

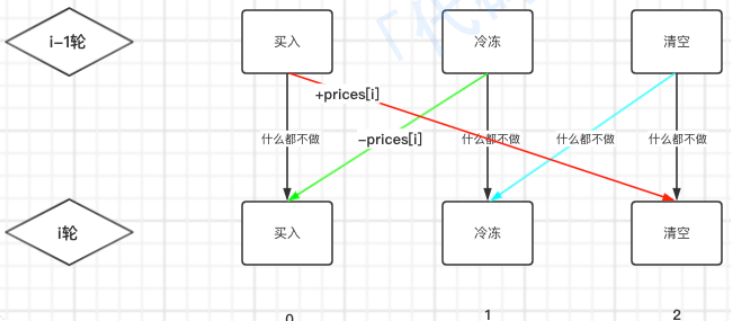










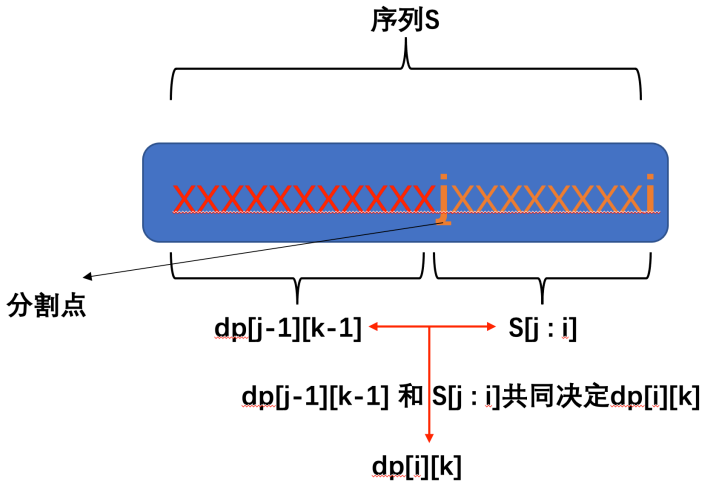


「代码随想录」知识星球

weikunkun

区间DP模型

> 给出一个序列，明确要求分割成K个连续区间，要求我们计算这些区间的某个最有性质




```
N = s.length();
K = group;
// 方便处理，有效字符的下标从1开始
// 前i元素构成k组的最优结果
dp =  [N+1][K+1];
for (int i = 1; i <= N; i++) {   // 遍历序列元素
    for (int k = 1; k <= min(i, K); k++) {   // 遍历组
        for (int j = 0; j <= i-1; j++) {   // 寻找最优的j（分割点）
            // 转移方程
            // 根据 dp[j-1][k-1] 和 S[j:i]求解
        }
    }
}
ans = dp[N][K];
```

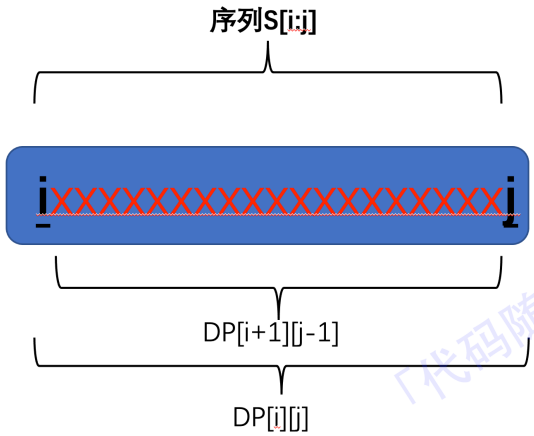
「代码随想录」知识星球

weikunkun

相关题目：

LC813 最大平均和的分组
LC1278 分割回文串 III
LC410 分割数组的最大值
LC1335. 工作计划的最低难度

> 第二种区间DP模型



1. dp[i][j]: 对于s[i:j]序列取最优结果
2. 思路：dp[i][j]通过dp[i+1][j-1]等小区间获得
3. **一定要明确好初始值**

```
初始化;
for (int len = 2; len <= N; len++) { // 先遍历区间大小
    for (int i = 0; i + len - 1 < N; i++) { // 再遍历起始位置（初始位置）
        int j = i + len - 1; // 右端点位置
        // 进行状态转移
        // 具体的状态转移还是要根据题意来
        // 下面的属于一种情况
        // 1. 在[i:j]区间找到最优的结果
        for (int k = i; k <= j; k++) {
            dp[i][j] = best();
        }
    }
}

return dp[0:N];
```

「代码随想录」知识星球

weikunkun

适用情景：针对整个序列，经过操作，获取最优解。

感觉这种类型的题目普遍有些复杂，模型并不特别适用，定义好dp数组，状态的转移的确是从小区间到大区间，但具体怎么筛选，还是需要根据题目具体展开。而且感觉整体难度对于个人来说偏高

整体思路：按照区间大小，依次枚举整个区间的所有子区间的结果，最后整个区间的最优结果是从这些子区间中的最优结果逐步结算得出的（怎么突然感觉是贪心呢？？？难道贪心属于动归的特例？？？）

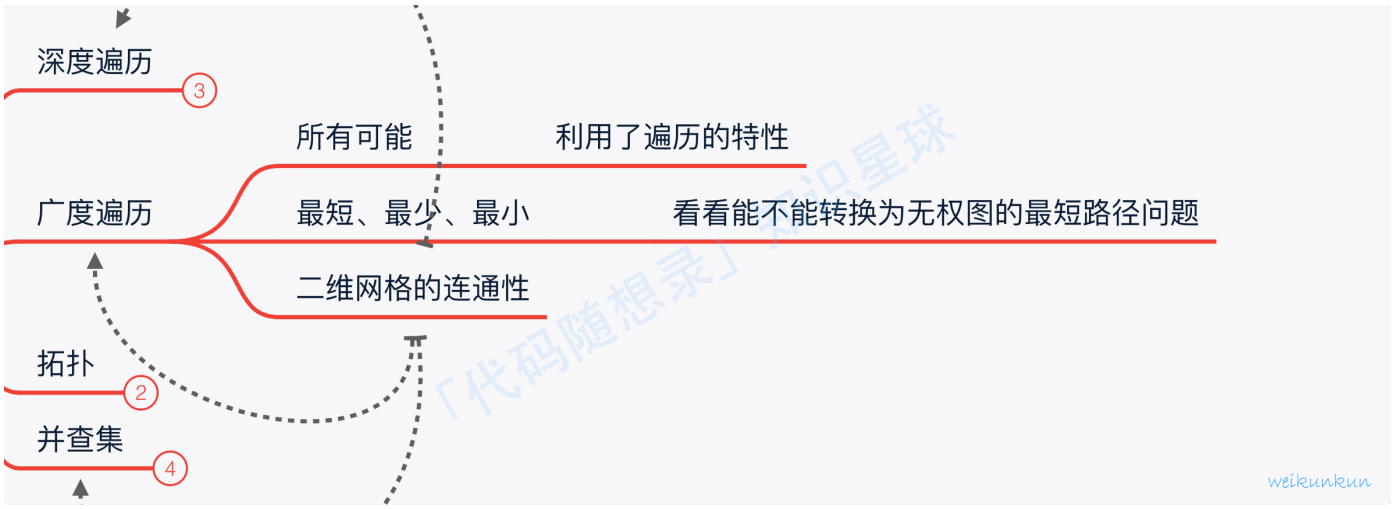
相关题目：

664. Strange-Printer
1547. 切棍子的最小成本
1682. 最长回文子序列 II(plus会员)
1690. 石子游戏 VII
1745. 回文串分割 IV
1000. 合并石头的最低成本

664. 奇怪的打印机

1. 确定dp数组含义和下标：dp[i][j] 为 [i, j] 区间的最小打印次数
2. 递推公式：对于区间，可以从边界开始考虑
    1. 当s[i] == s[j]时，打印 s[i] 时顺便就可以打印 s[j] ，中间的区域是可以被覆盖的，所以只考虑边界，只需要打印一次 i 位置 ，j 位置也被顺便打印完成，即等于 j 位置不计算的次数，所以 dp[i][j] = dp[i][j - 1]
    2. 当s[i] != s[j]时，可以分为两个区间，[i, k]和[k + 1, j]， k 取 [i, j)， 此时只要找到两个区间d打印次数之和最小的次数即可，即 dp[i][j] = min(dp[i][k] + dp[k + 1][j])
3. 初始化：考虑到区间长度为1，即 i == j 时，dp[i][i] = 1
4. 遍历顺序：i 从大到小，j 从小到大，因为在 s[i] != s[j] 的情况下，dp[i][j] 依赖于 dp[i][k], dp[k + 1][j]
    1. 如果 i 从 0 开始，k 从 i 开始，那么 对于dp[k + 1][j] ，左边界取的是 i + 1，此时可能就没有被遍历过
    2. 如果 i 从 n - 1 开始，k 从 i 开始，那么 对于dp[k +1][j]，左边界取的是 i + 1，此时 i 是倒序遍历，所以 dp[k + 1][j] 肯定被遍历过了
5. j 同理，只有从 i + 1开始，k 从 i 开始，dp[i][k]才会被遍历过，保证无后效性

BFS



一些实现细节：

1. 是否访问的标识一定要处理好，不然就炸了
2. 根据题意，选择使用邻接矩阵、邻接表存储图（偏向邻接表）
3. 和背包模型一样，要学会抽象

编辑距离

1. 这几道题给定的是两个字符串，
2. 要区分是等价为求 子序列（不连续） 还是 子数组（连续）
3. 还有个比较新颖的点是 dp 数组中 用 dp[i][j] 来表征 i-1 、j-1 相关的信息

思维导图

	子序列之编辑距离		
题型		**初始化**	**dp含义以及递推式**
删除长串	392.判断子序列	根据dp的含义均初始化为0	dp[i][j]：以i-1结尾字符串和以j-1结尾的字符串，相同子序列的**长度** s[i-1] == t[j-1]： **dp[i][j] = dp[i-1][j-1] + 1** 不相等：**dp[i][j] = dp[i][j-1]**//只删除长串t 结果：最终判断dp[s.size()][t.size()] 与字符串s的长度是否相等
	115.不同子序列	t不为空时，s子串为空时：dp[i][0] = 1 t为空时，s子串不为空时：dp[0][j] = 0	dp[i][j]：以i-1为结尾的s子序列中出现以j-1为结尾的t的**个数** **s[i-1] == t[j-1]:dp[i][j] = dp[i-1][j-1] + dp[i-1][j]** **[不使用s[i-1]进行匹配！！！]** **dp[i][j] = dp[i-1][j]**//只删除长串s 结果：dp[s.size()][t.size()] 为最终不同子序列的个数
两个字符串均可删除	583.两个字符串的删除	word1不为空时，word2为空时：for(int i = 0;i <= word1.size();i++) dp[i][0] = i; word1为空时，word2不为空时：for(int j = 0;j <= word2.size();j++) dp[0][j] = j;	dp[i][j]：以i-1为结尾的字符串word1，和以j-1位结尾的字符串word2，想要达到相等，所需要删除元素的**最少次数** 相等：**dp[i][j] = dp[i-1][j-1]** 不相等：**dp[i][j] = min(dp[i-1][j],dp[i][j-1]) + 1**
	72.编辑距离【比583.多了一步替换操作】		dp[i][j] 表示以下标i-1为结尾的字符串word1，和以下标j-1为结尾的字符串word2，最近编辑距离 相等：**dp[i][j] = dp[i-1][j-1]** 不相等：**dp[i][j] = min(dp[i-1][j-1],min(dp[i-1][j],dp[i][j-1])) + 1**
总结： dp的定义式：vector<vector<int>> dp(word1.size() + 1,vector<int>(word2.size() + 1))；增加考虑空串，便于初始化以及递推式 递推式分两步：比较word1[i-1]与word2[j-1]是否相等；①相等时的递推式② 不相等的递推式			

回文串

1. 区分是求 回文子序列（不连续）还是 回文子串（连续）
2. 这个类型题中的 dp 数组，需要把一个字符串从两个维度来分析，展成一个二维的，有点不太习惯，需要画图做题适应一下。
3. 这个二维图推断的时候，都是左下角的部分不参与，最后只有右上角参与了判断。
4. 这个题型的 dp 数组，是把两个维度当做一个闭区间进行判断，因为是闭区间，所以天生就有 i <= j

(1) 判断回文子串时，设为 bool 类型，判断区间内是否是回文子串
(2) 判断回文子序列时，设为 int 类型，用来存储区间内的最长回文子序列，这里的初始化也很特别，i=j 时初始化为 1， 其余为0

5. dp 数组的遍历顺序也很巧妙

(1) 回文子串时，i 是从 s.size()-1 到 0，j 是从 i 到 s.size()-1
(2) 回文子序列时，i 是从 s.size()-1 到 0，j 是从 i+1 到 s.size()-1，这里为啥是 i+1 呢，因为 i=j 已经初始化时赋值了，我们就是要根据这个初始化来进行接下来的所有推理呀

思维导图

<table>
<tr><th colspan="4">子序列之回文串问题</th></tr>
<tr><th colspan="2">题型</th><th>初始化</th><th>递推式</th></tr>
<tr><td rowspan="2">回文子串</td><td>647.回文子串</td><td rowspan="2">全部初始化为false；默认均不是</td><td rowspan="2">当s[i] == s[j]时；
if(j-i <=1 || dp[i+1][j-1]) dp[i][j] = true//区间中要么元素个数小于等于1；要么内部区间为真
当为true的情况，作相应操作：
• 统计回文子串个数
• 更新最大回文子串长度 以及对应回文子串起始位置</td></tr>
<tr><td>5.最长回文子串
可拓展求长度</td></tr>
<tr><td>回文子序列</td><td>517.最长回文子序列</td><td>区间中只有一个元素的位置赋值为1(dp[i][i])；其余赋值为0</td><td>s[i] == s[j] dp[i][j] = dp[i+1][j-1] + 2;
不相等： dp[i][j] = max(dp[i+1][j],dp[i][j-1]);</td></tr>
<tr><td colspan="4">回文子串总结：
1.布尔类型的dp[i][j]：表示区间范围[i,j] （注意是左闭右闭）的子串是否是回文子串，如果是dp[i][j]为true，否则为false；vector<vector<bool>> dp(s.size(),vector<bool>(s.size(),false));
2.同样分两种情况讨论：s[i] 与s[j]是否相等；但是只需要处理相等的；不相等肯定不是；</td></tr>
</table>

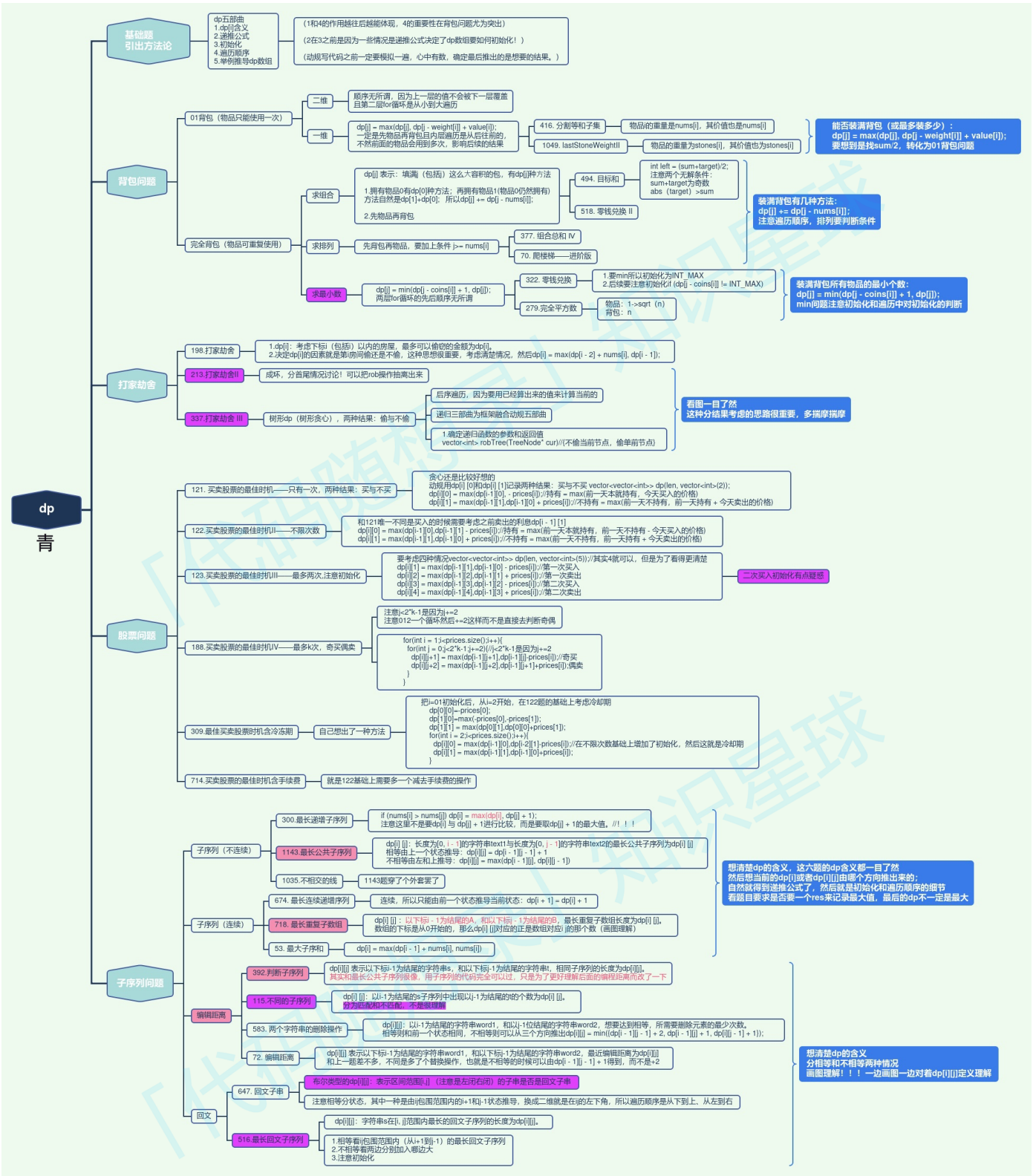
DP递推公式总结

动态规划各类问题的 dp 数组特点　　本人

	dp 数组维度	dp 数组定义特点	解决的问题	最终结果与 dp 数组的关系
背包问题	一维 二维		求最值	最终的结果为 dp 数组下标的最大值: dp[n]或 dp[n][m]
打家劫舍	一维 二维	dp[i]:第 i 天偷窃金额的最大值，考虑 nums[i]，但不一定包含		
股票问题	二维	dp[i][j]:第 i 天 j 状态下的利润最大值，考虑 nums[i]，但不一定包含		
子序列问题	一维 二维	当遍历到 dp[i][j]或 dp[i]时，无论是字符串还是数组，必须包含 str[i]或 nums[i]。	求最值 求子串 求子序列	对于数组:最终结果为 dp 数组所有元素的最大或最值:res=max(res, dp[i])
				对于字符串:最终的结果为 dp 数组下标的最值: dp[n]或 dp[n][m]

DP题目总结

原图链接：「代码随想录」知识星球



排序算法

1、插入排序的思路在链表的排序中更加直观

2、运用快速排序中的每次的选择基准值的思路可以解决的问题

1. 求出现次数超过一半的元素
2. 第K个元素

3、归并排序排序合并阶段还可以解决的问题

1. 求逆序对的个数
2. 求右侧小于当前元素的个数

4、堆排序

因为每次的堆顶元素是当前序列的最值，所以可以理解为优先队列，然后可以和类似任务处理、topK问题、贪心里面的区间问题和霍夫曼树、Prim算法生成最小生成树、Dijkstra算法的优化

其他算法

经典面试算法

1. 如何仅用递归函数和栈操作逆序一个栈

1、题目要求

一个栈依次压入1、2、3、4、5，那么从栈顶到栈底分别为5、4、3、2、1。将这个栈转置后，从栈顶到栈底为1、2、3、4、5，也就是实现栈中元素的逆序，但是只能用递归函数来实现，不能用其他数据结构;

2、解答

想要不使用别的数据结构完成栈的反转，就必须借助递归函数的返回值来实现这些操作。首先一个递归要深入栈中，取得并移除栈底元素，但是不应该改变除了栈底外的其他元素的顺序。；第二个递归是要将取出的元素按要求再放入栈中，push操作要在递归返回后再push。也就完成了要求;

3、总结

对于容器来说，想要改变元素的顺序，同时不适用额外的数据结构，可以借助递归函数的返回值来合理的安排你想要的操作。

2. 猫狗队列

1、题目要求：

实现一种狗猫队列的结构，可以完成add，pollall，polldog，pollcat等操作;

2、解答：

本题考查实现特殊数据结构的能力以及针对特殊功能的算法设计能力，略过。

3. 打印两个有序链表的公共部分

1、题目要求：

给定两个有序链表的头指针head1和head2，打印两个链表的公共部分;

2、解答：

循环比较两个指针的数值，两个指针都不能为空，小的推进，相等就记录并两个指针都推进。

4. 分别用递归和非递归方式实现二叉树先序、中序和后序遍历

1、解答：

递归方式很简单，就是调整处理节点和递归左右子节点的顺序。

而非递归方式，就是要利用栈来模拟递归的过程。如果想使用统一的模板来实现三种遍历，中序遍历是一个问题，它的访问顺序和处理顺序是不一致的。解决办法是使用一个空节点来标记访问。即在取得栈中的一个节点非空时，栈中依次推入该节点和空节点，表示这次是访问，并不处理。如果取栈中的元素为空，就表示这次要弹出栈中的下以哦个非空元素，表示这次是处理这个节点。这样就将三种遍历的不统一的访问和处理顺序统一起来。当然，此时要注意的是，不能将空节点压栈，否则回导致算法失效。

5. 斐波那契数列问题的递归和动态规划

1、解答：

方法一：

暴力递归--即return f(n - 1) + f(n - 2),这是2的n次方级别的时间复杂度;

方法二：

遍历--从左到右依次计算，直到计算到第n项，这是n级别的空间复杂度;

方法三：

矩阵乘法--F（n）=F（n-1）+F（n-2），是一个二阶递推数列，一定可以用矩阵乘法的形式表示，且状态矩阵为2×2的矩阵，所以，求斐波那契数列第N项的问题就变成了如何用最快的方法求一个矩阵N次方的问题，这是一个logn级别的空间复杂度（提示，将N次方转化为位的形式，一个矩阵用于保存累乘的结果，一个矩阵用于计算矩阵的1，2，4，8...次方，也就是记录每一位对应的矩阵乘法结果。

大数据处理问题

1. 公共数据问题

策略：大而化小，分而治之的策略

方法：哈希函数；根据hash值映射到对应小文件；然后依次比较小文件

1、文件A、B各有2T的文件，文件按照行存放了的字符串；找到A、B两个文件中公共的字符串

步骤：

1. 根据内存限制划分文件数目 N；尽量保证每个文件分配的数据是均匀的；
2. 根据哈希值绑定特定文件；遍历A文件；对每一行做hash函数计算；根据哈希值将该行数据映射到一个小文件【A1-AN中的某一个】；遍历文件B做同样的处理；会得到对应的B1-BN个文件
3. 逐个比较<A1,B1>文件对【不对应的小文件不可能有相同的字符串】；此时数据量足够小，可以装载到hashmap中进行对比，最后得到结果

拓展：

可能会出现多个文件映射到同一哈希值，如何解决？改变哈希函数；每次遇到冲突映射到小文件则改变哈希函数

2、A,B两个文件各存放50亿url，每个为64Byte，限制内存4G找出公共url

步骤：

1. 根据内存限制划分文件数目 1000，每个文件的大小是300M；尽量保证每个文件分配的数据是均匀的；
2. 根据哈希值绑定特定文件；遍历A文件；对每一行的URL做hash函数计算；根据哈希值将该行数据映射到一个小文件【A1-AN中的某一个】；遍历文件B做同样的处理；会得到对应的B1-BN个文件【A/B两个文件使用相同的哈希函数，保证文件之间的一一对应】
3. 逐个比较<A1,B1>文件对【不对应的小文件不可能有相同的URL】；此时数据量足够小，可以装载到hashmap中进行对比，最后得到结果

拓展：

可能会出现多个文件映射到同一哈希值，如何解决？改变哈希函数；每次遇到冲突映射到小文件则改变哈希函数

3、1000w有重字符串，对字符串去重

步骤：

1. 【映射到不同文件】将字符串根据哈希函数映射到不同的文件中
2. 【单独文件找重复】逐个文件使用哪个hash_set检查是否有重复的字符串
3. 【重复之中找交集】然后对所有文件操作后的hash_set求交集

2. TOP N问题

策略：

哈希映射差分成小文件，在每个小文件中利用hash_map统计出现频率最大的数据；然后将所有文件中的最大数据放到一个大根堆中，从大根堆中取N个元素即可
如何在海量数据中找出重复最多一个

方法：

通过hash映射为小文件，通过hash_map统计各个小文件重读最多的并记录次数，对每个小文件重复最多的进行建立大根堆

1、上亿有重数据，统计最多前N个

步骤：

（1）内存存不下

1. 通过hash映射为小文件
2. 通过hash_map统计各个小文件重读最多的并记录次数
3. 对每个小文件重复最多的进行建立大根堆并重复N次取走堆顶并重建堆操作

（2）内存存得下

1. 直接内存通过hash_map统计并建大根堆
2. 重复N次取走堆顶并重建堆操作

2、海量日志数据，提取出某日访问百度次数最多的那个IP（同1）

步骤：

1. 将IP % 1000映射到1000个小文件中
2. 相同IP会被映射到同一个文件
3. 不会出现累加和更大情况
4. 分1000次在内存处理小文件，得到频率最大IP（使用map统计）
5. 对这1000个IP建立大根堆

3、1000w查询串统计最热门10个（同2）

同上

4、1G的文件，里面1行1个不超过16字节的词。内存限制1M，返回频数最高前100（同2）

步骤：

1. 将单词 % 5000存入5000小文件
2. 平均各文件约200K
3. 对超过1M的文件继续分割直到小于200K
4. 使用map统计各个词出现的频率
5. 对5000词使用堆排序或归并排序

3. 分布式的TOP N问题

1、分布在100台电脑的海量数据，统计前十。

步骤：

1. 各数据只出现在一台机器中
    1. 先在独立机器得到前十
    2. 若可以放入内存直接堆排序
    3. 若不可全放入内存：哈希分块 -> map统计 -> 归总堆排
    4. 再将100台计算机的TOP10组合起来堆排序
2. 同一元素可同时出现在不同机器中

- 遍历所有数据，重新hash取模，使同一个元素只出现在单独的一台电脑中，然后采用上面方法先统计每台电脑TOP10再汇总起来

Astar算法

1. Astar算法简介

1. 当前拥有一个地图，地图上面有起点和终点，还有障碍物
2. 一个机器人在起点，希望用最短的距离到达终点
3. Astar算法可以用来解决这个问题

2. Astar数据结构 + 公式

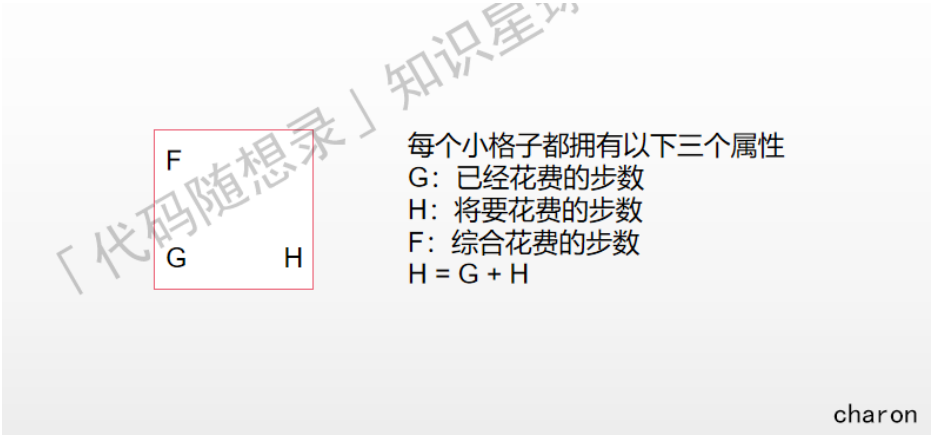
1、两个数据结构

```
1. Open表，用来存储当前能够到达的结点
2. Close表，用来存储已经到达过的结点
```

2、一个公式

（1）公式：F = G + H

每一个格子拥有三种属性，即：



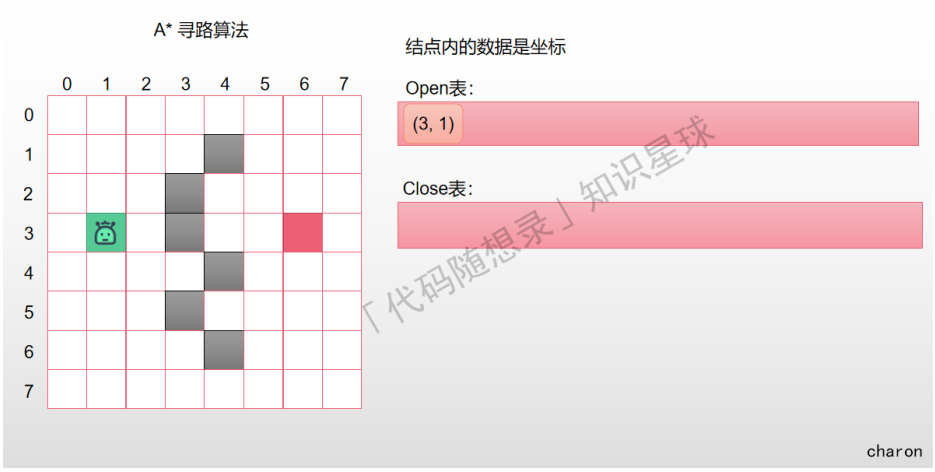
（2）三种属性的意义

1. G：从起始结点走到当前结点需要花费多少步（已经花了的步数）
2. H：不考虑障碍的情况下，从当前结点走到终点需要花费的步数（预估仍然需要花的步数）
3. F：对G和H的综合评估，也就是路径上如果拥有这个格子的话，最终需要花费的步数数量的预估

3. 算法步骤

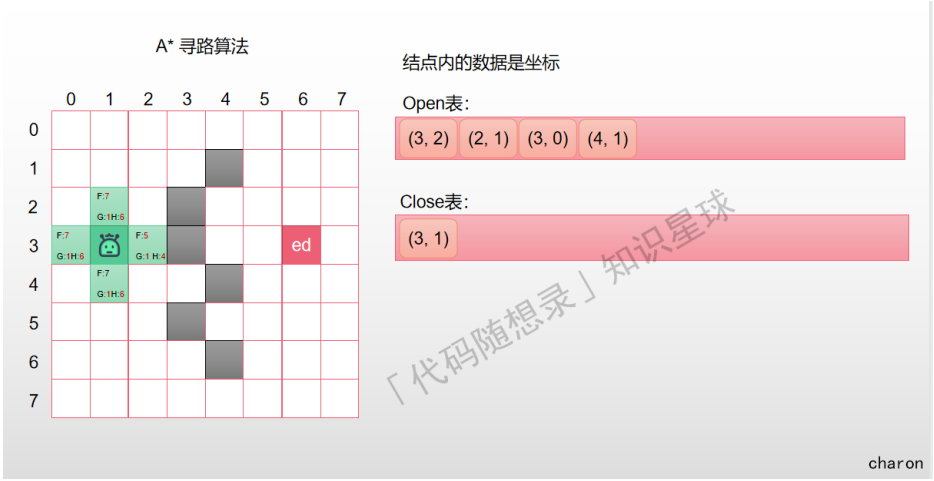
1、第一轮操作

1. 首先，我们需要将起点（机器人起始点）放到open表中

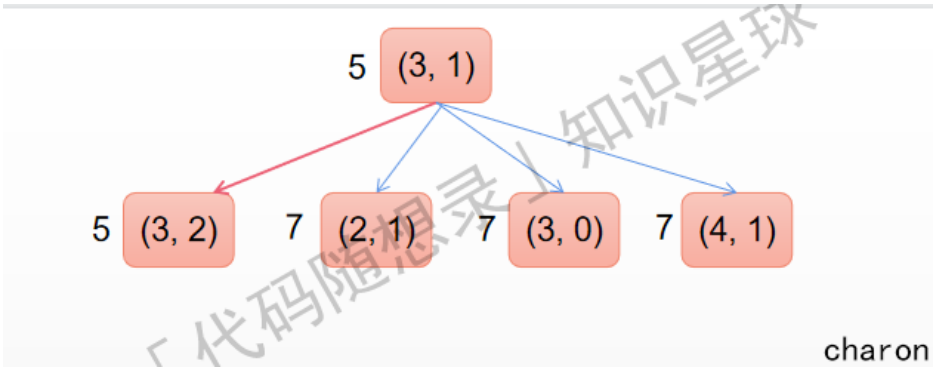


2. 在【Open表】中寻找出一个F值最小的结点作为【当前结点】，把【当前结点】移出【Open表】并且加入到【Close表】中
3. 对【当前结点】进行扩展，检查【当前结点】四周是否有能够到达的结点（扩展结点），如果这些【扩展结

点】不在【Open表】中或者【Close表】中，就把这些【扩展结点】计算出F值后加入到【Open表】中，并且把【当前结点】设置为这些【扩展结点】的父节点。



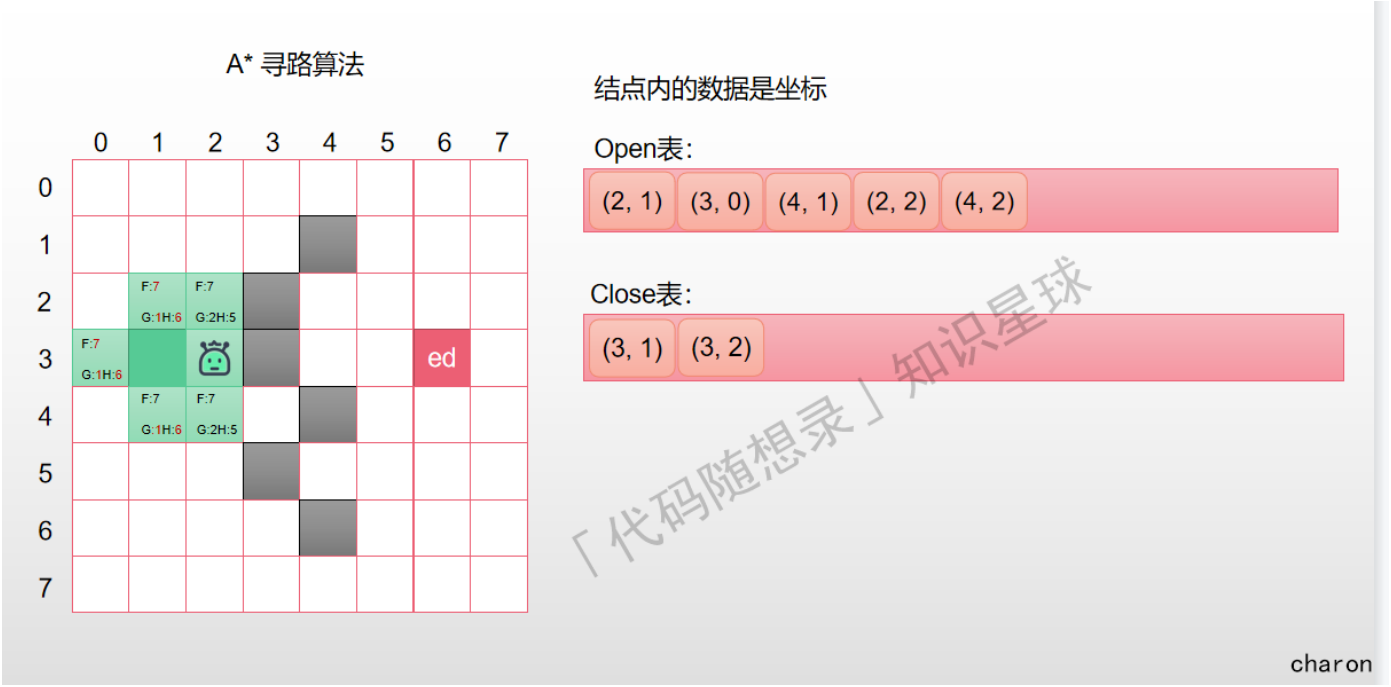
4. 当前的父子结点关系如下



2、第二轮操作

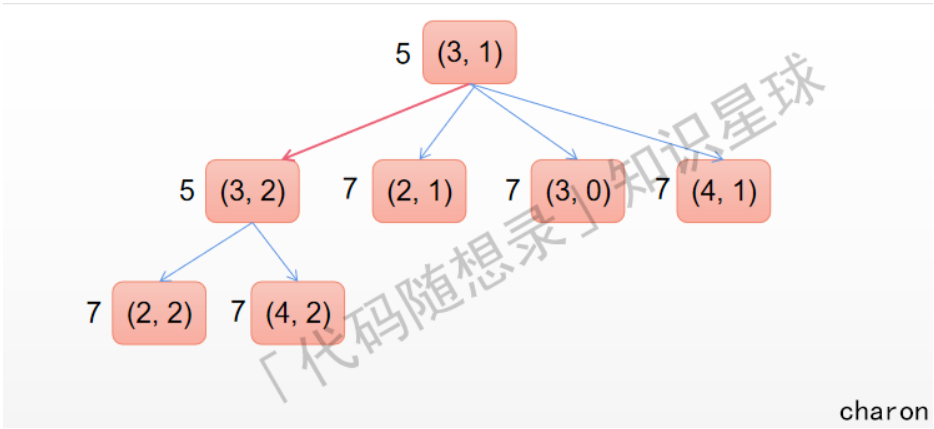
因为当前的【Open表】不为空并且没有找到【end结点】，所以重复第一轮的第二三步操作

1. 寻找【Open表】中F值最小的结点作为【当前结点】， 把【当前结点】移出【Open表】并且放入【Close表】，接下来是对【当前结点】进行扩展
2. 检查【当前结点】四周是否有能够到达的结点（扩展结点），如果这些【扩展结点】不在【Open表】中或者【Close表】中，就把这些【扩展结点】计算出F值后加入到【Open表】中，并且把【当前结点】设置为这些【扩展结点】的父节点。



3. 因为（3， 3）是【障碍物】，（3， 1）已经存在于【Close表】中，所以这两个结点都不加入【Open表】中

当前的父子结点关系如下：

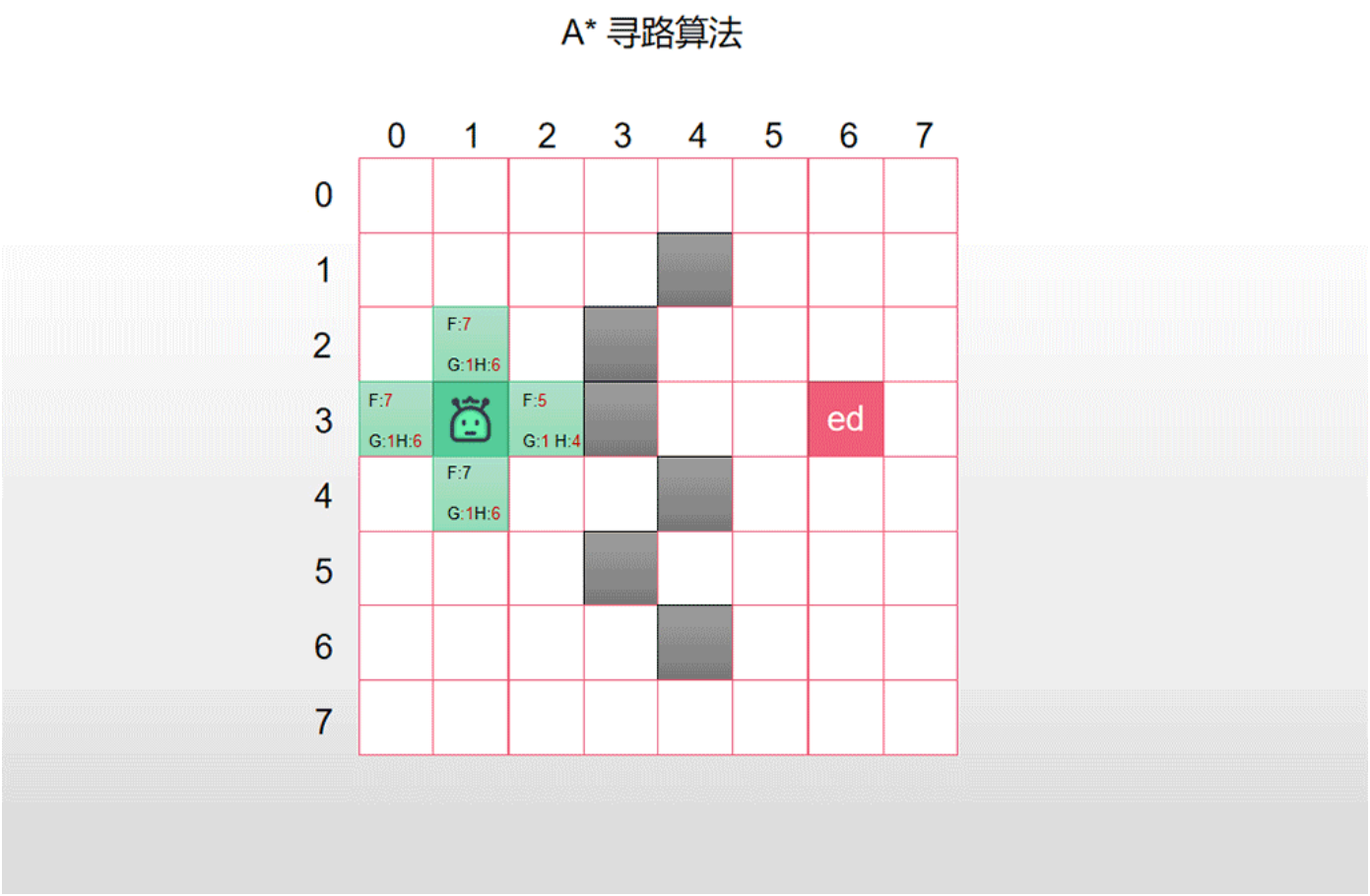


3、第三轮操作

接下来的方式就是重复以下步骤：

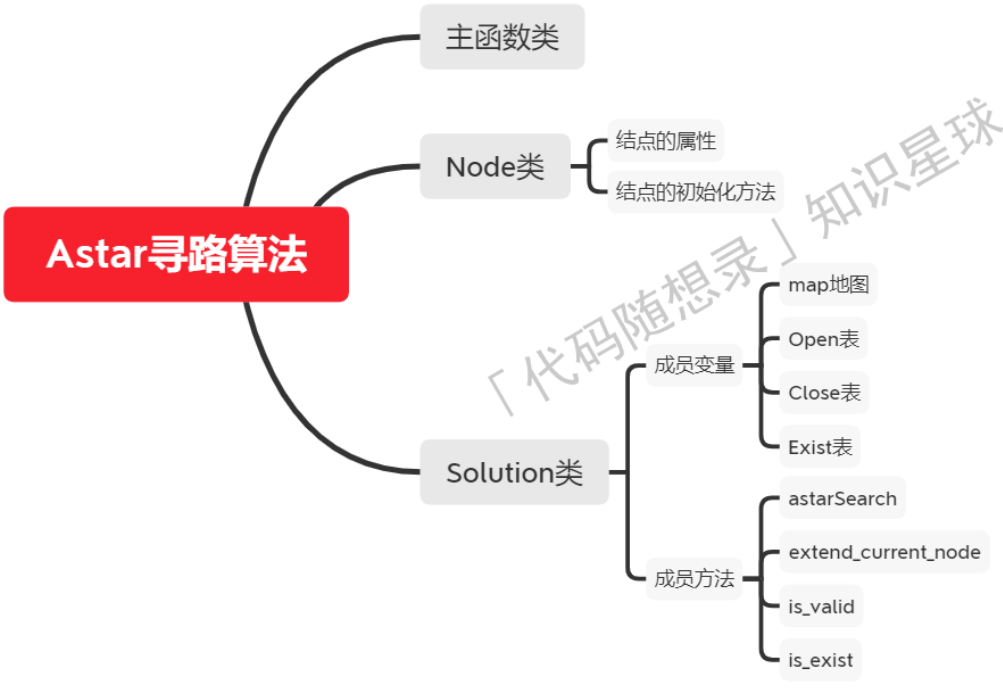
1. 从【Open】表中寻找F值最小的结点移入【Close表中】并且当作【当前结点】进行检查扩展
2. 将能够到达的新结点加入到【Open表】中
   下面是全部的过程的动图示例，仔细看F的值，每次都选择F值最小的结点进行检查扩展

原图链接：「代码随想录」知识星球



4. 代码实现

首先对整个算法我们需要整体上的认识：



charon

1. 因为每次都需要从Open表中寻找出最小的值，所以Open表可以使用小根堆实现，这里使用Java的优先队列实现小根堆的功能；
2. Java的优先队列不能迭代遍历，所以在检查检点是否出现过可以通过添加一个Exist表，这个表格将收集出现在Open表和Close表中出现过的所有结点
3. Close表使用普通的数组即可

需要一个Node类记录经过的每一个结点的信息，Node类的信息如下：

```
//结点的属性
//因为每个结点都需要存放在优先队列中，所以需要实现Comparable接口
class Node implements Comparable<Node> {
public int x; //x坐标
public int y; //y坐标
public int F; //F属性
public int G; //G属性
public int H; //H属性
public Node Father;  //此结点的上一个结点
//构造函数
public Node(int x, int y) {
    this.x = x;
    this.y = y;
}
//通过结点的坐标和目标结点的坐标可以计算出F， G， H三个属性
//需要传入这个节点的上一个节点和最终的结点
public void init_node(Node father, Node end) {
    this.Father = father;
    if (this.Father != null) {
        //走过的步数等于父节点走过的步数加一
        this.G = father.G + 1;
    } else { //父节点为空代表它是第一个结点
        this.G = 0;
    }
    //计算通过现在的结点的位置和最终结点的位置计算H值
    this.H = Math.abs(this.x - end.x) + Math.abs(this.y - end.y);
    this.F = this.G + this.H;
}
    // 用来进行和其他的Node类进行比较重写的方法
    @Override
    public int compareTo(Node o) {
        return Integer.compare(this.F, o.F);
    }
}
```

ps:为了方便数据的处理，所有的成员变量我都public了（这好吗？这不好！不要学我哈，还是要学会封装的。）

接下来是Solution类，所有的算法和使用到数据结构都存放在这个类中
首先需要一个地图：

```
// -1 -> 墙壁,  1 -> 起点 2 -> 终点
public int[][] map = {
    {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1},
    {-1, 0, 0, 0, 0, 0, 0, 0, 0, -1},
    {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
    {-1, 0, 0, 0, -1, 0, 0, 0, 0, -1},
    {-1, 0, 1, 0, -1, 0, 0, 2, 0, -1},
    {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
    {-1, 0, 0, 0, -1, 0, 0, 0, 0, -1},
    {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
    {-1, 0, 0, 0, 0, 0, 0, 0, 0, -1},
    {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1}
    };
```

这个map就是上面的地图，由于判断地图是否越界过于麻烦，添加了辅助区域，让我们对【当前结点】进行扩展操作的时候判断扩展是否越界变得简单直观，只要不等于-1就代表没有越界，而不必判断x坐标和y坐标的范围。

有了地图之后我们还需要【Open表】，【Close表】，【Exist表】
对结点进行扩展添加的时候除了需要判断结点是否合法，还需要判断结点是否在【Open表】和【Close表】中出现过
但是由于【Open表】不是可以遍历的数据结构，为了方便使用【Exist表】来记录当前结点是否出现在【Open表】中和【Close表】中

```
//Open表用来存放能够到达的结点
//Open表会自动把F值最小的结点放在队首
public PriorityQueue<Node> Open = new PriorityQueue<Node>();
//Close表用来存放已经到达的结点
public ArrayList<Node> Close = new ArrayList<Node>();
//Exist表用来存放两张表出现过的结点
public ArrayList<Node> Exist = new ArrayList<Node>();
```

判断一个结点是否出现过（is_exist方法）

```
public boolean is_exist(Node node)
{
    for (Node exist_node : Exist) {
    //如果这个结点在Exist中出现过，返回true
        if (node.x == existnode.x && node.y == existnode.y) {
        return true;
        }
    }
    //没有出现返回false
    return false;
}
```

怎么判断一个结点是否合法（is_valid方法）

```
public boolean is_valid(int x, int y) {
    // 如果结点的位置在地图上是-1，则不合法
    if (map[x][y] == -1) return false;
    for (Node node : Exist) {
        //如果结点出现过，不合法
        if (is_exist(new Node(x, y))) {
            return false;
        }
    }
    //以上情况都没有则合法
    return true;
}
```

怎么扩展【当前结点】的【上】【下】【左】【右】四个方向的结点(extendcurrentnode方法)

```
public ArrayList<Node> extendcurrentnode(Node current_node) {
    //获取当前结点的x，y
    int x = current_node.x;
    int y = current_node.y;
    //如果当前结点的邻结点合法，就加入到neighbour_node
    ArrayList<Node> neighbour_node = new ArrayList<Node>();
    if (is_valid(x + 1, y)) {
        Node node = new Node(x + 1, y);
        neighbour_node.add(node);
    }
    if (is_valid(x - 1, y)) {
        Node node = new Node(x -1, y);
        neighbour_node.add(node);
    }
    if (is_valid(x, y + 1)) {
        Node node = new Node(x, y + 1);
        neighbour_node.add(node);
    }
    if (is_valid(x, y - 1)) {
        Node node = new Node(x, y - 1);
        neighbour_node.add(node);
    }
    //返回合法的邻结点们
    return neighbour_node;
}
```

Astar寻路算法具体实现(astarSearch方法)

```
public Node astarSearch(Node start, Node end) {
//把第一个开始的结点加入到Open表中
    this.Open.add(start);
    this.Exist.add(start);
```

```
    //主循环
    while (Open.size() > 0) {
    //取优先队列顶部元素并且把这个元素从Open表中删除
    Node current_node = Open.poll();
    //将这个结点加入到Close表中
    Close.add(current_node);
    //对【当前结点】进行扩展，得到一个邻居结点数组
    ArrayList<Node> neighbournode = extendcurrentnode(currentnode);
        //对这个邻居数组遍历，看是否有目标结点出现
        for (Node node : neighbour_node) {
            if (node.x == end.x && node.y == end.y) {//找到目标结点就返回
                //init_node操作把这个邻居结点的父节点设置为当前结点
                //并且计算出G， F， H等值
                node.initnode(currentnode,end);
                return node;
            }
            if (!is_exist(node)) {
                //没出现过的结点加入到Open表中并且设置父节点
                //进行计算对G, F, H 等值
                node.initnode(currentnode, end);
                Open.add(node);
                Exist.add(node);
            }
        }
    }
    //如果遍历完所有出现的结点都没有找到最终的结点，返回null
    return null;
 }
```

5. 总体代码

```
import java.util.ArrayList;
import java.util.PriorityQueue;

public class Astar {
    public static void main(String[] args) {
        int[][] map = {
        {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1},
        {-1, 0, 0, 0, 0, 0, 0, 0, 0, -1},
        {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
        {-1, 0, 0, 0, -1, 0, 0, 0, 0, -1},
        {-1, 0, 1, 0, -1, 0, 0, 2, 0, -1},
        {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
        {-1, 0, 0, 0, -1, 0, 0, 0, 0, -1},
        {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
        {-1, 0, 0, 0, 0, 0, 0, 0, 0, -1},
        {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1}
        };
```

```
        Node start = new Node(4, 2);
        start.Father = null;
        Node end = new Node(4, 7);
        Solution solution = new Solution();
        Node res_node = solution.astarSearch(start, end);
        //渲染迷宫
        while (res_node != null) {
            map[res_node.x][res_node.y] = 11;
            res_node = res_node.Father;//迭代操作
        }

        for (int i = 0; i < 10; i++) {
            for (int j = 0; j < 10; j++) {
                System.out.printf("%3d", map[i][j]);
            }
            System.out.println();
        }
    }
}
//结点的属性
class Node implements Comparable<Node> {
    public int x; //x坐标
    public int y; //y坐标
    public int F; //F属性
    public int G; //G属性
    public int H; //H属性
    public Node Father;  //此结点的上一个结点
    //获取当前结点的坐标
    public Node(int x, int y) {
        this.x = x;
        this.y = y;
    }
//通过结点的坐标可以得到F,  G,  H三个属性
//需要传入这个节点的上一个节点和最终的结点
    public void init_node(Node father, Node end) {
        this.Father = father;
        if (this.Father != null) {
            this.G = father.G + 1;
        } else { //父节点为空代表它是第一个结点
            this.G = 0;
        }
        //计算通过现在的结点的位置和最终结点的位置计算H值
        this.H = Math.abs(this.x - end.x) + Math.abs(this.y - end.y);
        this.F = this.G + this.H;
    }
// 用来进行和其他的Node类进行比较
    @Override
    public int compareTo(Node o) {
        return Integer.compare(this.F, o.F);
```

```
    }
}

class Solution {
    //地图 -1 代表墙壁， 1代表起点，2代表终点
    public int[][] map = {
        {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1},
        {-1, 0, 0, 0, 0, 0, 0, 0, 0, -1},
        {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
        {-1, 0, 0, 0, -1, 0, 0, 0, 0, -1},
        {-1, 0, 1, 0, -1, 0, 0, 2, 0, -1},
        {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
        {-1, 0, 0, 0, -1, 0, 0, 0, 0, -1},
        {-1, 0, 0, 0, 0, -1, 0, 0, 0, -1},
        {-1, 0, 0, 0, 0, 0, 0, 0, 0, -1},
        {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1}
        };
    // Open表用优先队列
    public PriorityQueue<Node> Open = new PriorityQueue<Node>();
    //Close表用普通的数组
    public ArrayList<Node> Close = new ArrayList<Node>();
    //Exist表用来存放已经出现过的结点。
    public ArrayList<Node> Exist = new ArrayList<Node>();

    public Node astarSearch(Node start, Node end) {
        //把第一个开始的结点加入到Open表中
        this.Open.add(start);
        //把出现过的结点加入到Exist表中
        this.Exist.add(start);
        //主循环
        while (Open.size() > 0) {
            //取优先队列顶部元素并且把这个元素从Open表中删除
            Node current_node = Open.poll();
            //将这个结点加入到Close表中
            Close.add(current_node);
            //对当前结点进行扩展，得到一个四周结点的数组
            ArrayList<Node> neighbour_node = extend_current_node(current_node);
            //对这个结点遍历，看是否有目标结点出现
            //没有出现目标结点再看是否出现过
            for (Node node : neighbour_node) {
                if (node.x == end.x && node.y == end.y) {//找到目标结点就返回
                    node.init_node(current_node,end);
                    return node;
                }
                if (!is_exist(node)) { //没出现过的结点加入到Open表中并且设置父节点
                    node.init_node(current_node, end);
                    Open.add(node);
                    Exist.add(node);
                }
```

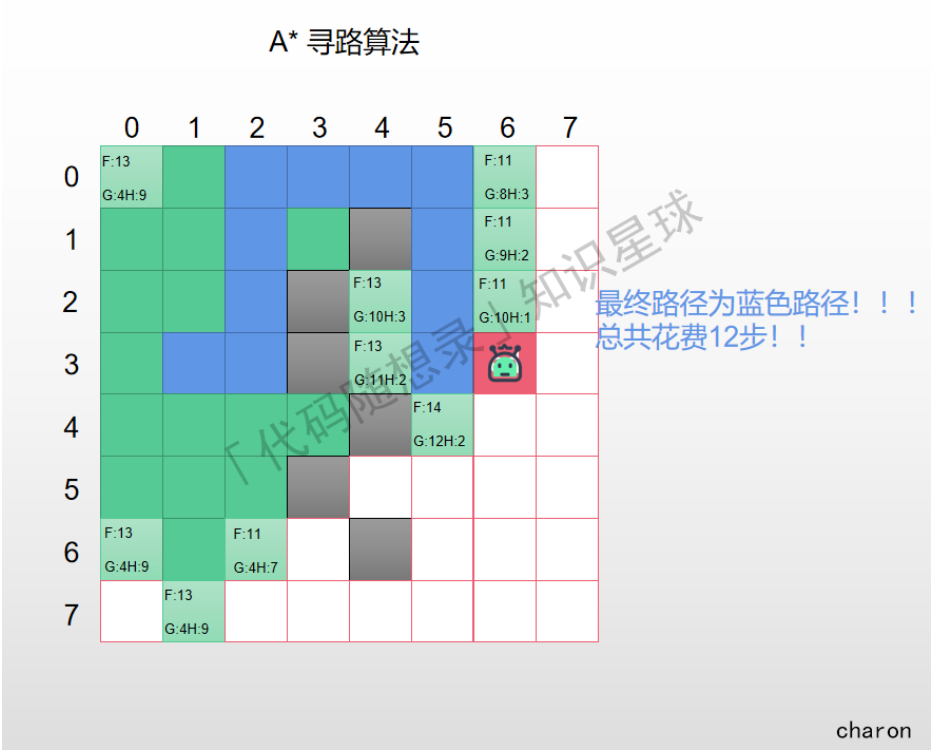
```
            }
        }
        //如果遍历完所有出现的结点都没有找到最终的结点，返回null
        return null;
    }

    public ArrayList<Node> extend_current_node(Node current_node) {
        int x = current_node.x;
        int y = current_node.y;
        ArrayList<Node> neighbour_node = new ArrayList<Node>();
        if (is_valid(x + 1, y)) {
            Node node = new Node(x + 1, y);
            neighbour_node.add(node);
        }
        if (is_valid(x - 1, y)) {
            Node node = new Node(x -1, y);
            neighbour_node.add(node);
        }
        if (is_valid(x, y + 1)) {
            Node node = new Node(x, y + 1);
            neighbour_node.add(node);
        }
        if (is_valid(x, y - 1)) {
            Node node = new Node(x, y - 1);
            neighbour_node.add(node);
        }
        return neighbour_node;
    }

    public boolean is_valid(int x, int y) {
        // 如果结点的位置是-1，则不合法
        if (map[x][y] == -1) return false;
        for (Node node : Exist) {
            //如果结点出现过，不合法
            //      if (node.x == x && node.y == y) {
            //        return false;
            //      }
            if (is_exist(new Node(x, y))) {
                return false;
            }
        }
        //以上情况都没有则合法
        return true;
    }

    public boolean is_exist(Node node)
    {
        for (Node exist_node : Exist) {
            if (node.x == exist_node.x && node.y == exist_node.y) {
```

```
                    return true;
                }
            }
            return false;
        }
    }
```

最终输出结果如下：

```
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1
-1  0  0  0 11 11 11 11  0 -1
-1  0  0 11 11 -1  0 11  0 -1
-1  0  0 11 -1  0  0 11  0 -1
-1  0 11 11 -1  0  0 11  0 -1
-1  0  0  0  0 -1  0  0  0 -1
-1  0  0  0 -1  0  0  0  0 -1
-1  0  0  0  0 -1  0  0  0 -1
-1  0  0  0  0  0  0  0  0 -1
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1
```

这是我们画的结果：



堆排序

1. 把无序数组构建成二叉堆
2. 循环删除堆顶元素，移到集合尾部，调节堆产生新的堆顶

```
void DownAdjust(vector<int>& A, int parentIndex, int len) {
    int temp = A[parentIndex];
    int childIndex = 2 * parentIndex + 1;
    while(childIndex < len) {
        if(childIndex + 1 < len && A[childIndex + 1] < A[childIndex])
            childIndex++;
        if(temp >= A[childIndex]) break;
            A[parentIndex] = A[childIndex];
            parentIndex = childIndex;
            childIndex = 2 * childIndex + 1;
    }
    A[parentIndex] = temp;
}
void HeapSort( vector<int>& A) {
    int len = A.size();
    if(len <= 1) return;

    for(int i = len / 2; i >= 0; i--) {
        DownAdjust(A, i, len);
    }

    for(int i = len  - 1; i > 0; i--) {
        int temp = A[i];
        A[i] = A[0];
        A[0] = temp;
        DownAdjust(A, 0 , i);
    }
}
```

最短路径问题

多源最短路径

得出图中任意两个顶点之间的最短路径

Floyd算法：

```
int dist[400][400];
void Floyd(int n)
{
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
}
```

Floyd本质上是一个动态规划的思想，每一次循环更新经过前k个节点，i到j的最短路径。

实际的思想就是，每次都考虑之前的最短路径，以及经过k节点，能否使得i→k +k→j的路径和小于i→j的路径。

初始化dist数组时，用int_max，i→i的路径填0。

算法本质就是暴搜，但是能搜出来就不错了

单源最短路

Bellman-Ford算法:

需要一个一维数组dist[]来存储每个点到起点的距离，设起点为S，终点为D，那这条最短路一定是S->P1->P2->...->D的形式。

思想和上面的算法非常一致，就是暴力破解。

你想求出某点到其他所有点的最短路径：

第一次遍历，你求得了与固定点一步可达的点的结果，其余点的结果未知

第二次遍历，就得到了你求得了与固定点两步可达的点的结果。

依次类推，只要图中不存在环，你遍历节点个数-2次，一定会得到所有的最短路径结果。这里用的是链式前向星存图

```
void Bellman_Ford(int n, int m)
{
    for (int j = 0; j < n - 1; ++j)
        for (int i = 1; i <= m; ++i)
            dist[edges[i].to] = min(dist[edges[i].to], dist[edges[i].from] +
edges[i].w);
}
```

SPFA算法:

观察发现，Bellman-Ford算法第一次遍历时，可能更新的点只可能是S能直接到达的点。

然后下一次可能被更新的则是S能直接到达的点能直接到达的点。

SPFA算法正是利用了这种思想，并进行了优化。

借用队列这一数据结构，在每次对点操作时更新它能到达的点，并把队列中不存在的点加入队列，并出队操作结束的点，这样直到队列为空，也就实现了要求。

总结：

只让当前点能到达的点入队

如果一个点已经在队列里，便不重复入队

如果一条边未被更新，那么它的终点不入队

```
void SPFA(int s)  //s为固定点编号，inqueue数组判断节点是否在队列中
{
    queue<int> Q;
    Q.push(s);
    while (!Q.empty())
    {
        int p = Q.front();
        Q.pop();
        inqueue[p] = 0;
        for (int e = head[p]; e != 0; e = edges[e].next)
        {
            int to = edges[e].to;
            if (dist[to] > dist[p] + edges[e].w)
            {
                dist[to] = dist[p] + edges[e].w;
                if (!inqueue[to])
                {
                    inqueue[to] = 1;
                    Q.push(to);
                }
            }
        }
    }
}
```

然而它的时间复杂度不稳定，和上一种算法的思路是相同的，因此在最坏情况下，时间复杂度为O(mn)

Dijkstra算法：

Dij基于一种贪心的思想，我们假定有一张没有负边的图，首先，起点到起点的距离为0，对起点和它能直接到达的所有点进行更新。

因为没有负边，遍历起点直连的节点，更新达到的节点的dist，这时可以肯定，离起点最近的那个顶点的dist一定已经是最终结果，不可能有比它更小的结果了。

这时候找到了起点直达的最短路径的点，以这个点为开始，对它能到达的点进行更新，此时就确定了可以从起点和起点直达的最短路径的点开始的最短路径值，每次更新后都选剩下为确定点中dist最小的节点去更新，直到所有节点都确定下来。

如何取出离给定点最近的点？**优先级队列。**

在一个点被访问前，优先队列里会存储这个点被更新的整个历史，出队时是按照从路径从小到大的顺序出队，由于小路径的时候已经此节点出队并判断过了，因此大的路径出队时直接跳过这个节点的处理即可。

首先写一个结构体,用于保存最短距离和节点编号。

```
struct Polar{
    int dist, id;
    Polar(int dist, int id) : dist(dist), id(id){}
};
```

然后确定

```
struct cmp{
    bool operator ()(Polar a, Polar b){ // 重载()运算符，使其成为一个仿函数
        return a.dist > b.dist;     // 这里是大于，使得距离短的先出队
    }
};
priority_queue<Polar, vector<Polar>, cmp> Q;  //小顶堆
int vis[n] //建立是否访问过的节点判断数组
```

算法实现：

```
void Dij(int s)
{
    dist[s] = 0;
    Q.push(Polar(0, s));
    while (!Q.empty())
    {
        int p = Q.top().id;
        Q.pop();
        if (vis[p])
            continue;
        vis[p] = 1;
        for (int e = head[p]; e != 0; e = edges[e].next)
        {
            int to = edges[e].to;
            dist[to] = min(dist[to], dist[p] + edges[e].w);
            if (!vis[to])
                Q.push(Polar(dist[to], to));
        }
    }
}
```

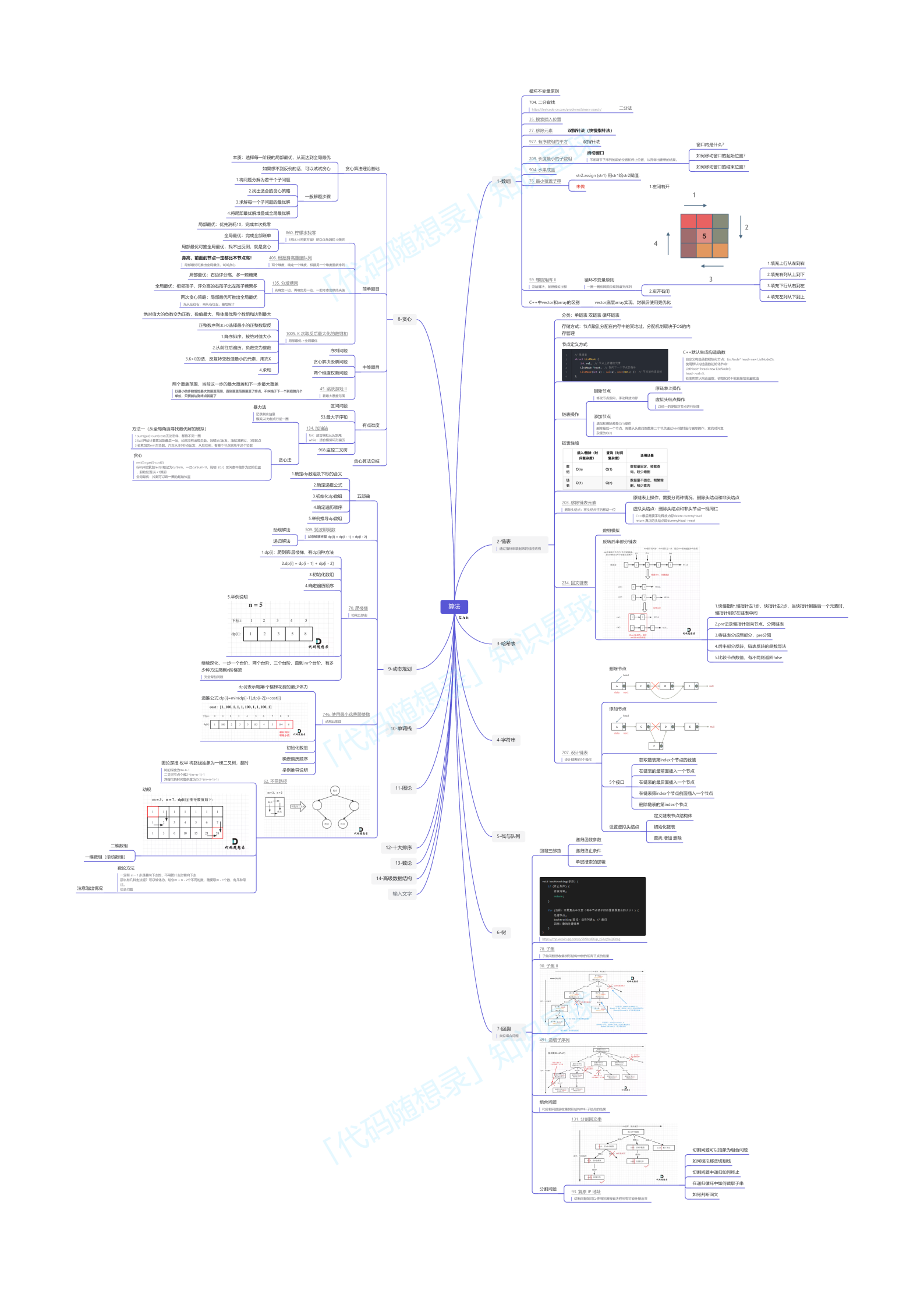
怎样打印路径?

只需要建立一个pre数组，在更新dist时候更新pre即可。由于算法最后的结果是一棵树，因此数组中保存此编号节点的父节点即可。每条路有且仅有一个祖先节点，一步步溯源上去的路径是唯一的。

思维导图

算法总结

原图链接：[「代码随想录」知识星球]

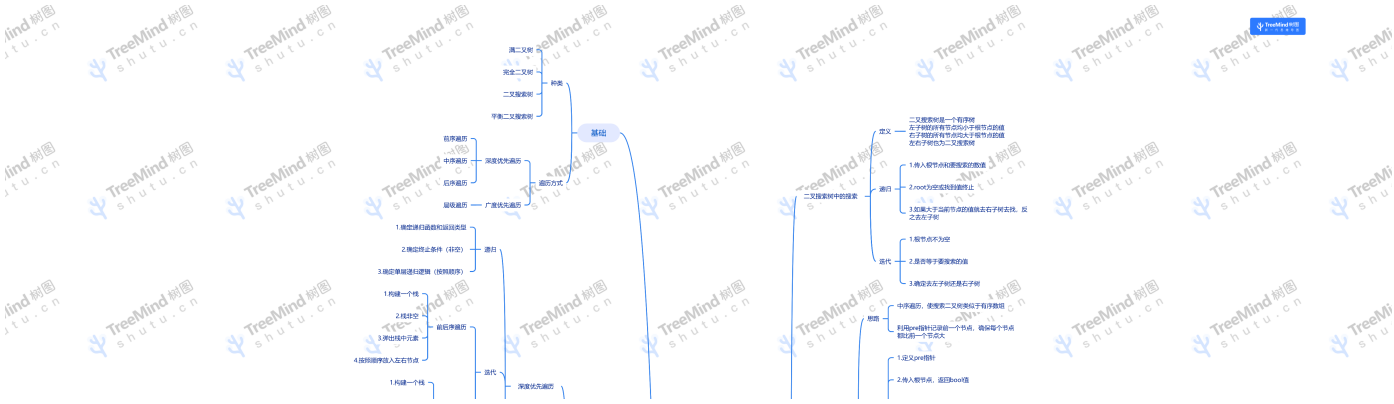

算法
1-数组
循环不变量原则
704. 二分查找
二分法
35. 搜索插入位置
27. 移除元素
双指针法（快慢指针法）
977. 有序数组的平方
双指针法
209. 长度最小的子数组
滑动窗口
窗口内是什么？
如何移动窗口的起始位置？
如何移动窗口的结束位置？
904. 水果成篮
76. 最小覆盖子串
str2.assign (str1) 用str1给str2赋值。
未做
1.左闭右开
1
2
3
4
5
59. 螺旋矩阵 II
循环不变量原则
一圈一圈给到固定规则每填充两个
2.左开右闭
1.填充上行从左到右
2.填充右列从上到下
3.填充下行从右到左
4.填充左列从下到上
C++中vector和array的区别
vector底层array实现，封装后使用更优化
2-链表
通过指针串联起来的线性结构
分类：单链表 双链表 循环链表
存储方式：节点散乱分配在内存中的某地址，分配机制取决于OS的内存管理
节点定义方式
C++默认生成构造函数
链表操作
删除节点
释放节点内存，手动释放内存
原链表上操作
虚拟头结点操作
以统一初始规则对节点进行处理
添加节点
链表性能
插入/删除（时间复杂度）
查询（时间复杂度）
适用场景
数组
O(n)
O(1)
数据量固定，频繁查询，较少增删
链表
O(1)
O(n)
数据量不固定，频繁增删，较少查询
203. 移除链表元素
删除头结点
原链表上操作，需要分两种情况，删除头结点和非头结点
虚拟头结点：删除头结点和非头节点一视同仁
C++最后需手动释放内存delete dummyHead
return 真正的头结点即dummyhead->next
234. 回文链表
数组模拟
反转后半部分链表
1.快慢指针:慢指针走1步，快指针走2步，当快指针到最后一个元素时，慢指针刚好在链表中间
2.pre记录慢指针倒向节点，分隔链表
3.将链表分成两部分，pre分隔
4.后半部分反转，链表反转的函数写法
5.比较节点数值，有不同返回false
707. 设计链表
设计链表的5个操作
删除节点
添加节点
5个接口
获取链表第index个节点的数值
在链表的最前面插入一个节点
在链表的最后面插入一个节点
在链表第index个节点前面插入一个节点
删除链表的第index个节点
设置虚拟头结点
初始化链表
定义链表节点结构体
直找 增加 删除
3-哈希表
4-字符串
5-栈与队列
6-树
7-回溯
类似组合问题
回溯三部曲
递归函数参数
递归终止条件
单层搜索的逻辑
78. 子集
子集问题是在树形结构中特别的所有节点的结果
90. 子集 II
491. 递增子序列
组合问题
对分割问题画出树形结构中对子串的处理
131. 分割回文串
分割问题
93. 复原 IP 地址
切割问题可以抽象为组合问题
如何模拟那些切割线
切割问题中递归如何终止
在递归循环中如何截取子串
如何判断回文
8-贪心
贪心算法理论基础
本质：选择每一阶段的局部最优，从而达到全局最优
如果想不到反例的话，可以试试贪心
一般解题步骤
1.将问题分解为若干个子问题
2.找出适合的贪心策略
3.求解每一个子问题的最优解
4.将局部最优解堆叠成全局最优解
简单题目
局部最优：优先消耗10，完成本次找零
全局最优：完成全部账单
860. 柠檬水找零
5元比10元更万能！所以优先消耗10美元
局部最优可推全局最优，找不出反例，就是贪心
身高，前面的节点一定都比本节点高！
406. 根据身高重建队列
局部最优：右边评分高，多一颗糖果
全局最优：相邻孩子，评分高的右孩子比左孩子糖果多
135. 分发糖果
两次贪心策略：局部最优可推出全局最优
绝对值大的负数变为正数，数值最大，整体最优整个数组和达到最大
正整数序列 K>0选择最小的正整数取反
1.降序排序，按绝对值大小
2.从前往后遍历，负数变为整数
3.K>0的话，反复转变数值最小的元素，用完K
4.求和
1005. K 次取反后最大化的数组和
中等题目
序列问题
贪心解决股票问题
两个维度权衡问题
两个覆盖范围，当前这一步的最大覆盖和下一步最大覆盖
以最小的步数增加最大的覆盖范围，直到覆盖范围覆盖了终点，不纠结于下一个到底跳几个单位，只要能达到终点就算了
45. 跳跃游戏 II
有难度
区间问题
53.最大子序和
134. 加油站
968.监控二叉树
贪心算法总结
暴力法
贪心法
方法一（从全局角度寻找最优解的模拟）
贪心
9-动态规划
五部曲
1.确定dp数组及下标的含义
2.确定递推公式
3.初始化dp数组
4.确定遍历顺序
5.举例推导dp数组
509. 斐波那契数
动规解法
递归解法
70. 爬楼梯
动规五部曲
1.dp[i]：爬到第i层楼梯，有dp[i]种方法
2.dp[i] = dp[i - 1] + dp[i - 2]
3.初始化数组
4.确定遍历顺序
5.举例说明
n = 5
746. 使用最小花费爬楼梯
动规五部曲
cost: [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
初始化数组
确定遍历顺序
举例推导说明
10-单调栈
11-图论
62. 不同路径
12-十大排序
13-数论
14-高级数据结构
输入文字
图论深搜 枚举 将路线抽象为一棵二叉树，超时
动规
二维数组
一维数组（滚动数组）
数论方法
注意溢出情况
组合问题

链表

原图链接：[链表题思维导图](#)

二叉树

> 原图地址：[二叉树思维导图](#)



二叉树

遍历方式

迭代 后序遍历

1. 构建队列
2. 队列不为空
3. 遍历当前队列元素
4. 遍历时向队列中插入当前元素的左右节点

迭代 层序遍历

1. 构建一个栈
2. 栈不为空
3. 当前节点不为空，指针遍历期间将放入元素
4. 中间节点收入栈后加一个null
5. 当前节点为空，栈弹出元素

统一迭代

2. 当前指针不为空或栈不为空
3. 将当前指针放入栈中，然后指向左节点
4. 遍历到最底层后，弹出元素，指向元素右节点

中序遍历

二叉树的属性

二叉树的最小深度

注意：最小深度是去叶子节点找，不是左右节点中一个为空就返回值

后序

1. 返回深度，属性值为节点
2. 节点为null时结束
3. 单层逻辑
   a. 求左右节点深度
   b. 左子树为空，右不为空，返回右节点深度，反之亦然
   c. 当左右子树都为空时，返回1

迭代

中间逻辑：判断节点是否为叶子并返回最小深度

前序

层序

二叉树的最大深度

后序

返回值为深度，单层逻辑为求左右节点最大深度

迭代

前序

1. 记录当前深度（全局变量）
2. 判断左节点是否为空
3. 将深度加一后递归
4. 深度减一（回溯）
5. 右节点重复上述操作

迭代

特殊点：根节点高度就是二叉树最大高度（故本题最简序均可）

对称二叉树

迭代

1. 通过函数参数left,right返回值bool
2. 确认终止条件：左右节点是否都为null，或是其中一个为null，或是值不相等
3. 传入外层节点和内层节点

迭代

1. 使用队列/栈
2. 队列是否为空
3. 弹出当今个节点开进行比较
4. 按顺序加入外层节点和内层节点

递归

采用后序遍历

完全二叉树的节点个数

普通二叉树的方式

层序遍历 —— 迭代

记录左右子树节点的数量 —— 递归

完全二叉树的方式

定义：
1. 只有最底层可能没填满
2. 最底层节点集中在左侧

情况一：满二叉树 —— 2 ^ depth - 1

情况二：最后一层叶子节点没有满 —— 递归到满二叉树的情况并返回节点数

向左遍历的深度是否等于向右遍历的深度 —— 判断是否为满二叉树

平衡二叉树

定义：一个二叉树每个节点左右两个子树高度差不大于1

递归

求高度：后序遍历

计算左右节点高度差是否大于1

迭代

遍历每个节点，将左子树和右子树高度进行比较

迭代实现深度

二叉树的所有路径

确认方法

记录路径 —— 前序遍历

路径记录后回溯

递归

1. 递归函数参数：记录路径path,存放结果集res,根节点root。
   返回值：无
2. 递归终止：是否为叶子节点
3. 单层递归逻辑：
   左节点不为空，递归，路径path删除最后一个元素（回溯和递归同级在一起）
   右节点亦然

迭代

左叶子之和

注意：是叶子节点，不是左侧节点

递归

返回左叶子的数值

注意要在父节点的单层逻辑上操作，因为只有父节点知道这是左叶子还是右叶子

迭代

前中后序均可

找树左下角的值

注意：是树的左下角，最底层的最左边
不是树的左节点

递归

前中后序均可，本题没有中间节点处理逻辑

全局变量记录最大深度

当深度第一个达到最大的节点就是要找的节点（要保证左节点优先遍历）

迭代

层序遍历（最简单） —— 迭代

路径总和

递归

1. 传入根节点，总和
2. 到达叶子节点，根据总和是否达到count返回boolean值
3. 分别对左右节点递归，并且将加上回溯

迭代

前中后序均可

创建一个栈，放节点本身和相对节点到该节点的路径总和

路径总和ii

递归

遍历整棵树，找到所有路径

注意递归回溯哦

迭代

要注意记录单条路径的path属性是什么

二叉搜索树的属性

验证二叉搜索树

递归

1. 节点等于null结束
2. 4.单层逻辑
   a.递归左边并记录返回的boolean值
   b.判断pre是否小于当前节点的值
   c.递归右边并记录返回的boolean值
3. 递归

迭代

二叉搜索树的最小绝对差

中序遍历

pre指针记录前一个节点

记录最小的差

二叉搜索树中的众数

普通二叉树

map记录出现的树与频率

排序

取最大频率

二叉搜索树

递归

1. 记录最大频率
2. 传入当前节点，不返回值
3. 当前节点为空时结束
4. 中序遍历
   a. technical左子树
   b. pre指针为null或当前指针与pre指针值不同，频率刷新
   c. 更新pre指针
   d. 如果当前频率等于最大频率，把当前指针的值放入结果集
   e. 如果当前频率大于最大频率，更新最大频率，清除结果集
   f. 递归右子树

迭代

将二叉搜索树转换为累加树

递归

反中序遍历（右中左）

pre指针记录前一个节点

迭代

二叉树公共祖先问题

二叉树的最近公共祖先

思路 —— 自底向上，后序遍历（回溯）

两种情况
1. 找到一个节点，左子树返回p，右子树返回q
2. 节点为p或q本身，有一个子孙节点为q或p

递归

1. 返回节点，传入当前节点，p和q
2. 当前节点等于p或q或为空结束
3. 单层逻辑
   a. 递归左子树并接收值，递归右子树并接收值
   b. 左侧两个递归返回值都不为空，则说明当前节点就是最近公共祖先，返回当前节点
   c. 如果有一个返回值，则返回那个值
   d. 如果没有，返回空
4. 总体效果

这里是遍历整棵树

迭代不适合

二叉搜索树的最近公共祖先

思路

前中后序均可，没有中节点的处理逻辑

节点在[p,q]则一定是p,q的公共祖先 —— 理解

p的祖先公共祖先的左子树
q的祖先公共祖先的右子树
如果还有一个节点在p,q的公共祖先，那就会使当前节点只有x或一个子节点就不能构成二叉树

递归

根据p,q判断要去当前节点左子树找还是去右子树找，找到后直接返回值

这里是只遍历一条边

迭代

二叉搜索树的修改与构造

二叉搜索树中的插入操作

递归

1. 参数是根节点和要插入的值，返回一个节点
2. 当前节点为空时，新建一个节点并返回
3. 根据插入值的大小去递归一条边并把返回值赋给当前节点对应的子树

迭代

用pre指针记录每一个节点

删除二叉搜索树中的节点

递归

1. 参数为当前节点和要删除的值，返回节点
2. 当节点为空时返回，说明树中没有要删除的节点
3. a.左右孩子都为空，直接删除节点，返回null
   b.删除的节点左孩子为空，有孩子不为空，返回右孩子
   c.bd的相反情况
   d.左右孩子都不为空，将左孩子放在右孩子的最左面的节点，返回右孩子

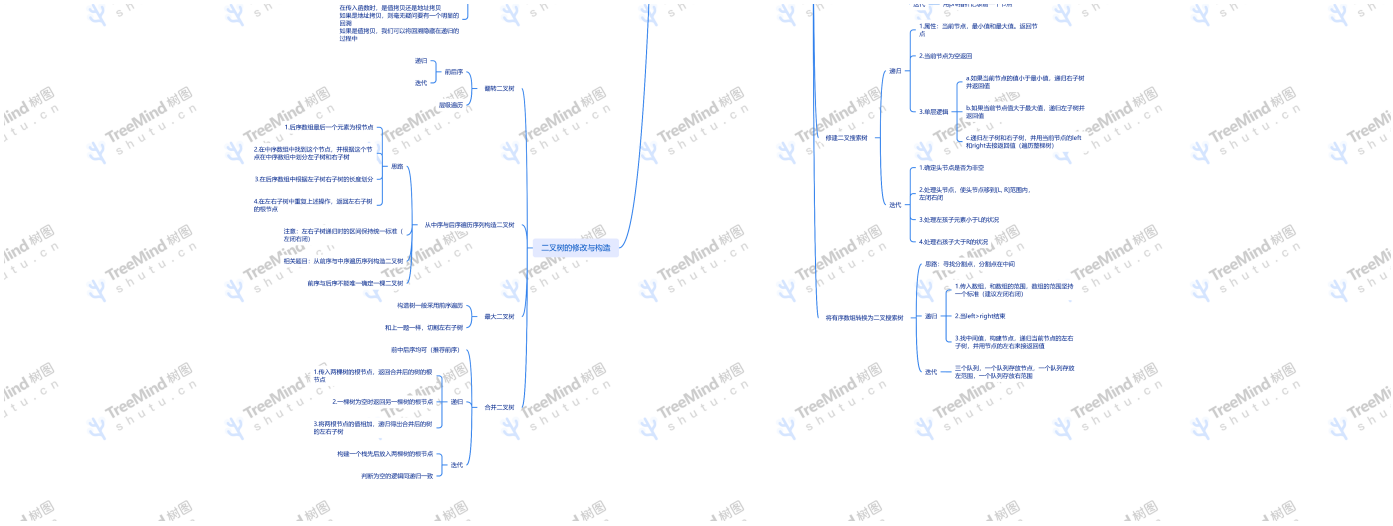
普通二叉树的删除方法

1. 遍历整棵树，找到要删除的节点
2. 如果右子树为空返回左子树
3. 将要删除的节点和其右子树的最左边的节点的值交换
4. 将右子树的最左面节点用null覆盖（在第二个步骤里已有体现，是一个优化）

二叉搜索树的修改与构造

递归

迭代

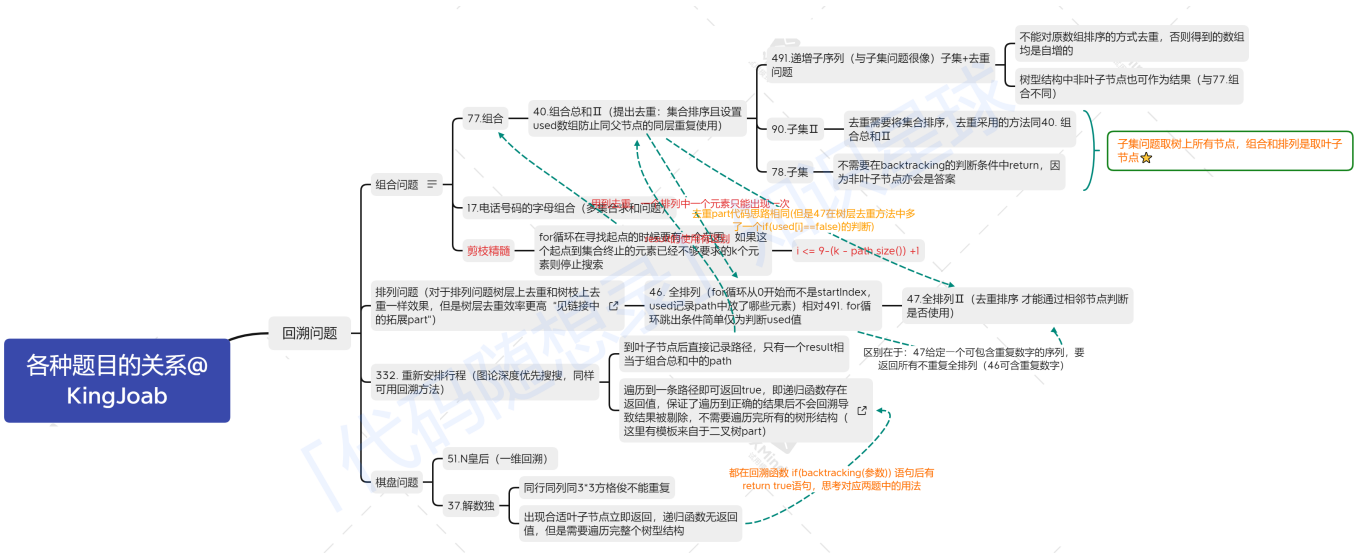


回溯

去重小结

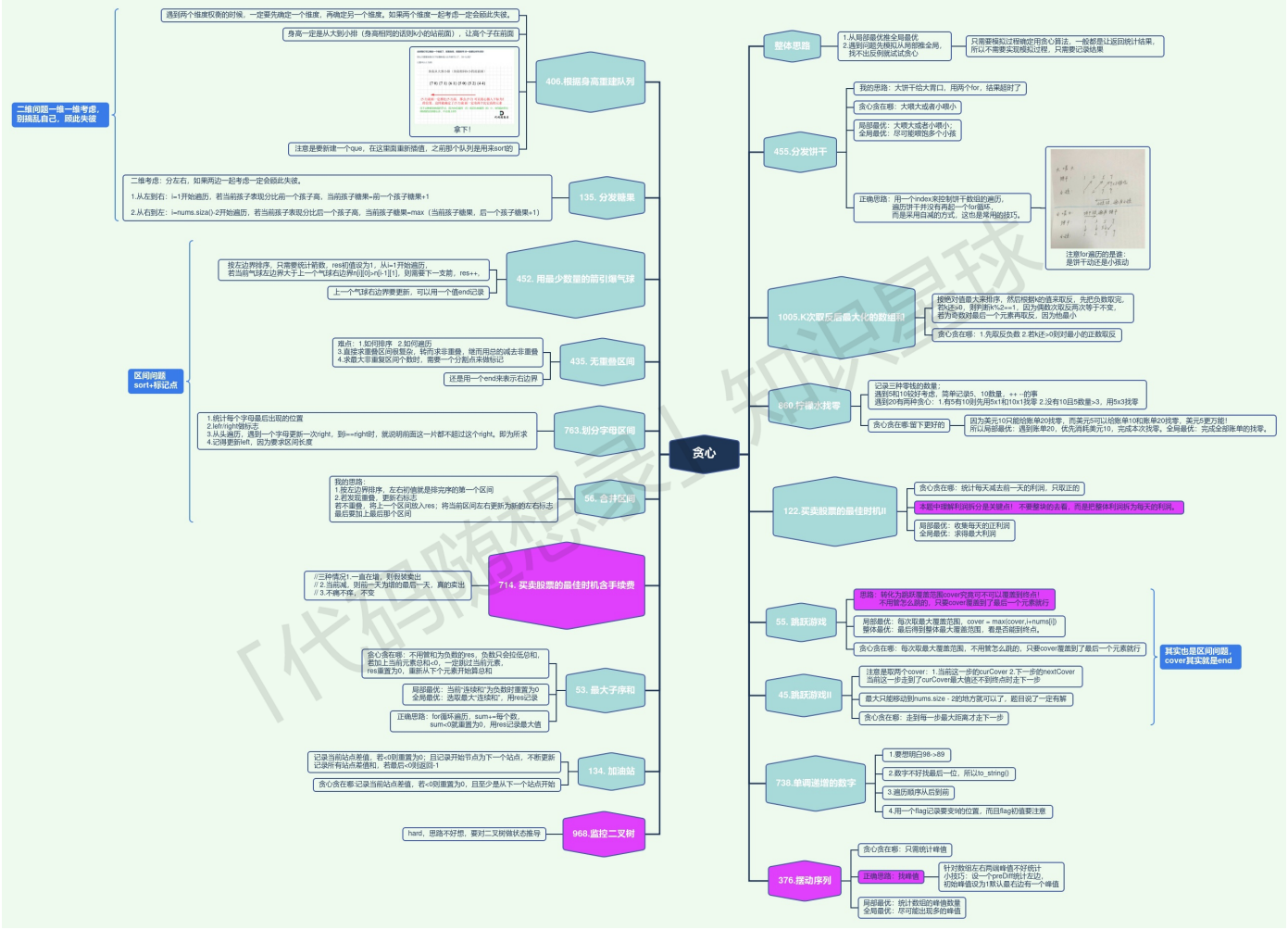
	StartIndex	Used[]数组	Flag[]数组
出现的原因	同一元素不能重复选取(下一层传入 i+1)	子集/组合问题中：出现了数值相同的多个元素，且同一层数值相同的多个元素不能被重复选取	用于解决递增子序列问题，由于递增子序列问题不能对数组进行排序，否则不满足原始题目要求，但又要求同一树层的元素不能重复选取，所以在每一层中都创建flag[]数组为该层去重
	同一元素可以被重复选取(下一层传入 i)	全排列问题中： 1. 同一层数值相同的多个元素不能被重复选取 2. 树枝的所有节点不能出现重复的元素(即使没出现数值相同的多个元素也需要 used[]数组)	
需要满足的特殊条件	无	当出现重复元素时数组要进行排序	无
适用场景	组合问题，子集问题，递增子序列问题	组合问题，子集问题，排列问题	递增子序列问题

本人



贪心

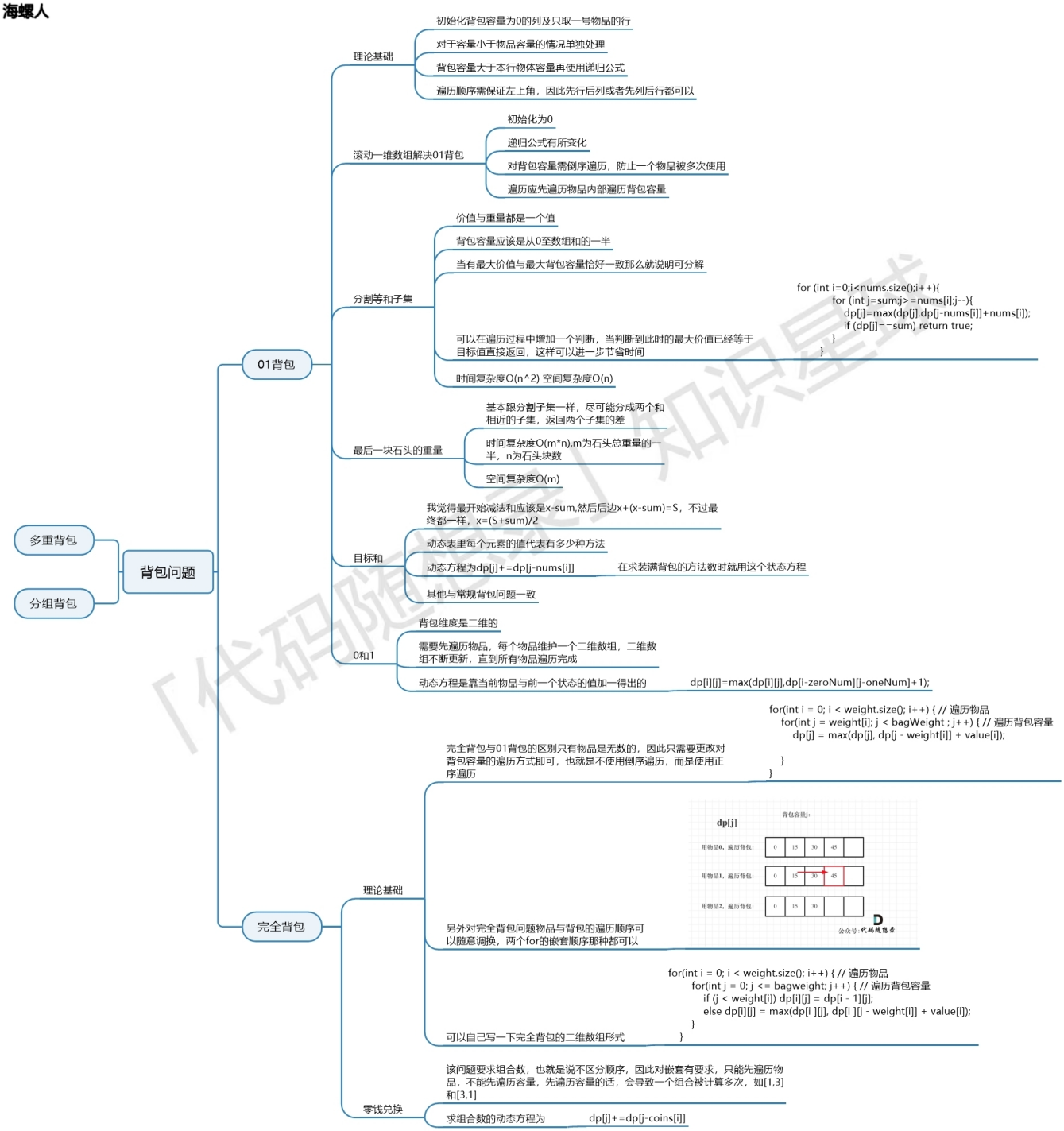
原图链接：「代码随想录」知识星球



背包

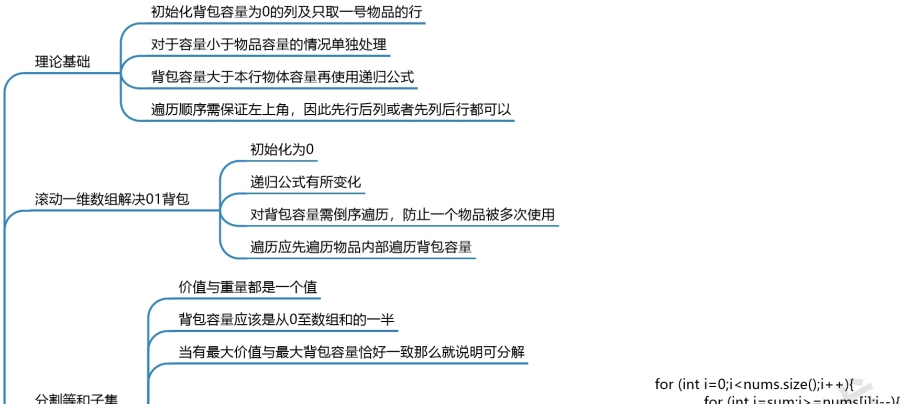
原图链接：「代码随想录」知识星球

海螺人

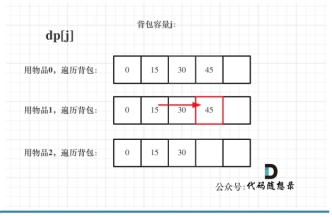


原图链接：「代码随想录」知识星球

海螺人



- 背包问题
  - 01背包
    - 可以在遍历过程中增加一个判断，当判断到此时的最大价值已经等于目标值直接返回，这样可以进一步节省时间
      ```
      dp[j]=max(dp[j],dp[j-nums[i]]+nums[i]);
      if (dp[j]==sum) return true;
      }
      }
      ```
    - 时间复杂度O(n^2) 空间复杂度O(n)
    - 最后一块石头的重量
      - 基本跟分割子集一样，尽可能分成两个和相近的子集，返回两个子集的差
      - 时间复杂度O(m*n),m为石头总重量的一半，n为石头块数
      - 空间复杂度O(m)
    - 目标和
      - 我觉得最开始减法和应该是x-sum,然后后边x+(x-sum)=S，不过最终都一样，x=(S+sum)/2
      - 动态表里每个元素的值代表有多少种方法
      - 动态方程为dp[j]+=dp[j-nums[i]]　在求装满背包的方法数时就用这个状态方程
      - 其他与常规背包问题一致
    - 0和1
      - 背包维度是二维的
      - 需要先遍历物品，每个物品维护一个二维数组，二维数组不断更新，直到所有物品遍历完成
      - 动态方程是靠当前物品与前一个状态的值加一得出的　dp[i][j]=max(dp[i][j],dp[i-zeroNum][j-oneNum]+1);
  - 完全背包
    - 理论基础
      - 完全背包与01背包的区别只有物品是无数的，因此只需要更改对背包容量的遍历方式即可，也就是不使用倒序遍历，而是使用正序遍历
        ```
        for(int i = 0; i < weight.size(); i++) { // 遍历物品
            for(int j = weight[i]; j < bagWeight ; j++) { // 遍历背包容量
                dp[j] = max(dp[j], dp[j - weight[i]] + value[i]);

            }
        }
        ```
      - 另外对完全背包问题物品与背包的遍历顺序可以随意调换，两个for的嵌套顺序那种都可以

      - 可以自己写一下完全背包的二维数组形式
        ```
        for(int i = 0; i < weight.size(); i++) { // 遍历物品
            for(int j = 0; j <= bagweight; j++) { // 遍历背包容量
                if (j < weight[i]) dp[i][j] = dp[i - 1][j];
                else dp[i][j] = max(dp[i-1][j], dp[i ][j - weight[i]] + value[i]);
            }
        }
        ```
    - 零钱兑换
      - 该问题要求组合数，也就是说不区分顺序，因此对嵌套有要求，只能先遍历物品，不能先遍历容量，先遍历容量的话，会导致一个组合被计算多次，如[1,3]和[3,1]
      - 求组合数的动态方程为　dp[j]+=dp[j-coins[i]]
    - 组合总和
      - 该问题的解是有顺序的，所以需要更改遍历顺序，应该把背包容量放在嵌套的外边，物品种类放在嵌套的里边
      - 另外存在过程的某两个数相加大于int的最大值，但是题干说了组合数在int范围内，因此对于那个大于范围的动态数组值选择不处理
    - 爬楼梯
      - 求上到n阶楼梯，每步可走1-m层，求有多少种方法
      - 是一个完全背包问题，每个步长看作物品种类可以多次使用，目标值就是n,解需要考虑顺序，因此是外背包容量，内物品种类。
      - 另外需注意嵌套循环是从1开始的，一位不存在0阶
    - 零钱兑换2
      - 各种硬币数量无限，求凑成总钱数的最小硬币数
      - 动态数组代表总钱数j的最少硬币数
      - 递推公式：dp[j] = min(dp[j - coins[i]] + 1, dp[j])
      - 初始化dp[0]=0
      - 遍历顺序无所谓
    - 完全平方数
      - 给定正整数 n，找到若干个完全平方数（比如 1, 4, 9, 16, ...）使得它们的和等于 n。你需要让组成和的完全平方数的个数最少。
      - 背包容量是n，物品种类是若干平方数，求组成和的个数最少
      - 动态方程：dp[j] = min(dp[j - i * i] + 1, dp[j]);
    - 单词拆分　有些没看懂动态公式！
  - 多重背包
    - 多个物品种类，每个物品种类都有不同数量个，因此可以把一个种类物品的数量拆开成每一个物品，这样就变成了01背包

	重量	价值	数量
物品0	1	15	1
物品0	1	15	1
物品1	3	20	1
物品1	3	20	1

	重量	价值	数量

「代码随想录」知识星球

物品0	1	15	2
物品1	3	20	3
物品2	4	30	2

物品1	3	20	1
物品2	4	30	1
物品2	4	30	1

背包遍历顺序

背包遍历顺序

本人

友情提示：动手画出各种情况下的 dp 数组有助于加深理解		两层 for 循环的遍历顺序	一层 for 循环的遍历方向（正序/倒序）	
		先选择物品 or 先选择背包	初始化时的遍历方向	循环递推 dp 数组时的遍历方向
01 背包	二维 dp[i][j]	都可以 理由：dp[i][j]总是由 dp 数组左上角的值推导得到	倒序遍历 for j:bagWeight to weight[0] dp[0][j] = dp[0][j - weight[0]] + value[0]; 理由：为了保证在每个重量的背包中物品 i 只被放入一次	两层循环均为正序遍历 理由：dp[i][j]总是由 dp 数组左上角的值推导得到
	一维 dp[j]	先选择物品后选择背包 理由：如果先背包后物品任意重量的背包只会选择其中一个价值最大的物品	无 for(j:1 to bagWeight) dp[j]=0 一般初始化为 0 即可	遍历物品时：正序遍历 遍历背包时：倒序遍历 倒序遍历理由：为了保证物品 i 只被放入一次

「代码随想录」知识星球

完全背包遍历顺序：

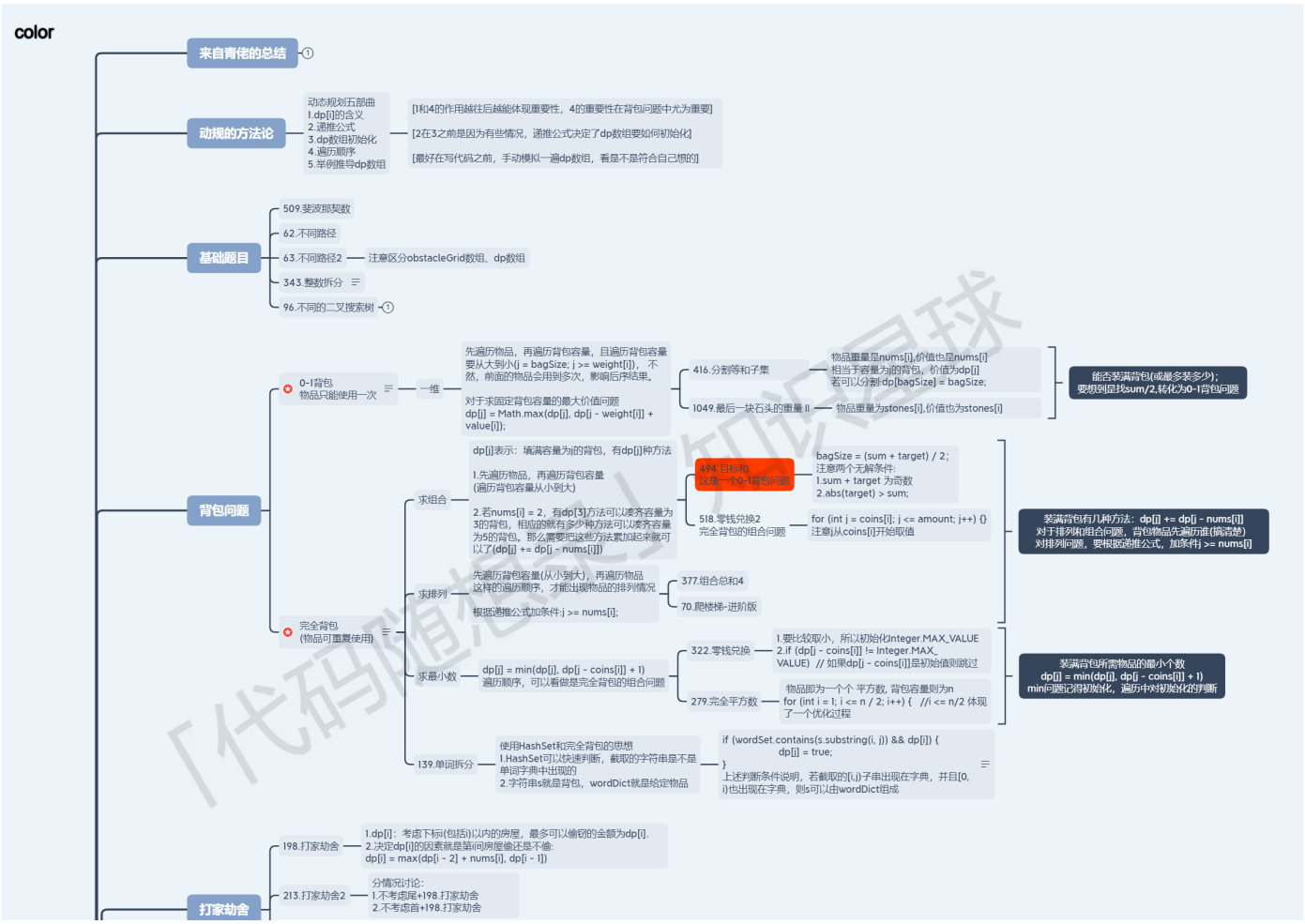
完全背包遍历顺序

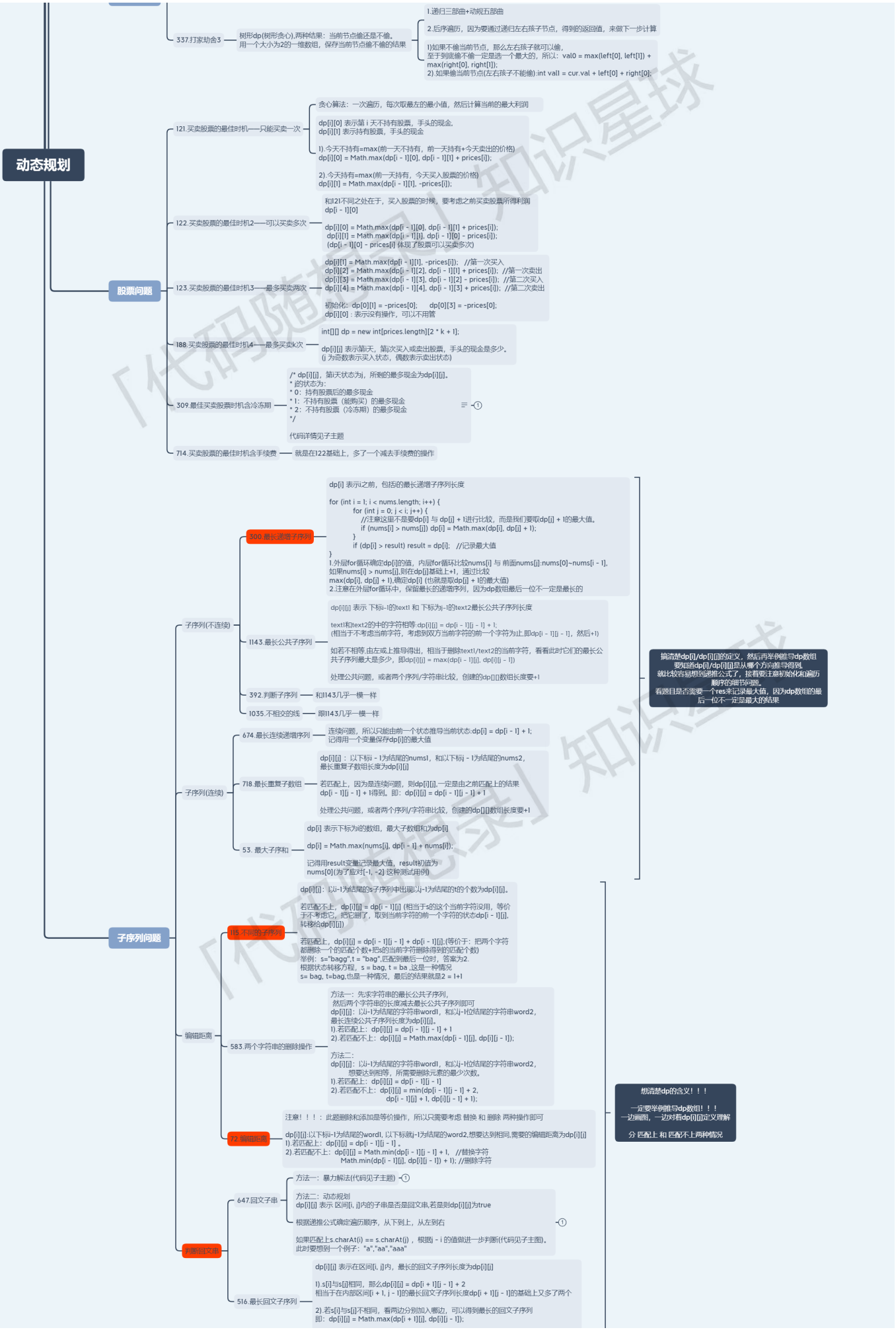
注:leetcode 中排列问题也有可能出现组合的字眼(如:377. 组合总和 Ⅳ),但是如果看到这句话: “顺序不同的序列被视作不同的组合”,则就是排列问题。

一维背包 dp[j]	纯完全背包	完全背包-组合	完全背包-排列
两层循环中每一层循环的遍历方向	均为正序遍历,即 i++, j++		
	理由:与 01 背包倒序遍历内层循环不同,完全背包的物品是可以添加多次的,所以需要正序遍历。		
两层循环的遍历次序(先背包 or 先物品)	都可以 (只要不涉及排列和组合先物品和先背包都不影响遍历次序)	先物品后背包	先背包后物品
	理由: dp[j]由 dp[j]之前的元素推导得出。	理由:先物品后背包,计算出来的才是组合数	理由: 先背包物品,计算出来的才是排列数 本人

动态规划

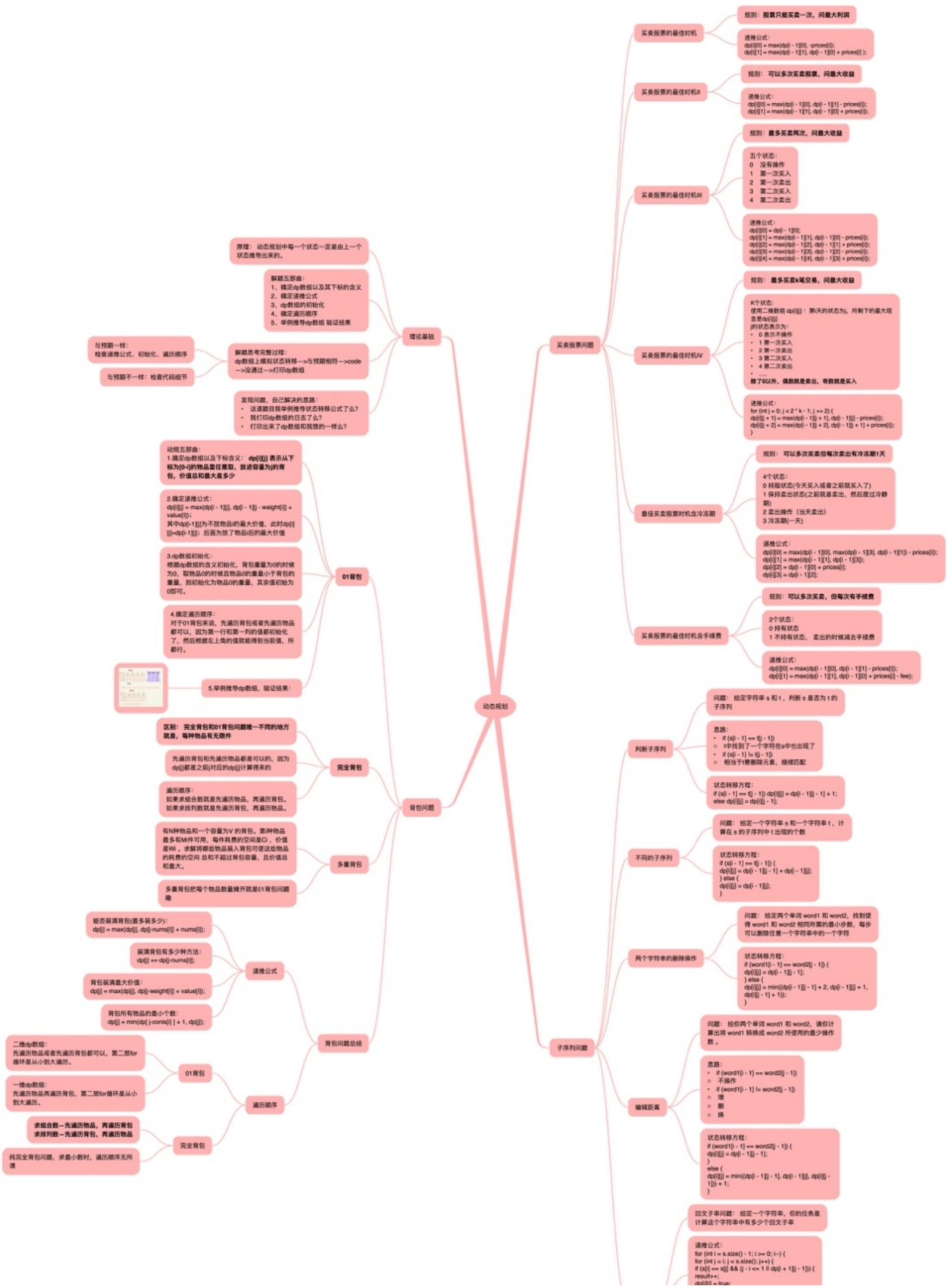
原图链接: 「代码随想录」知识星球

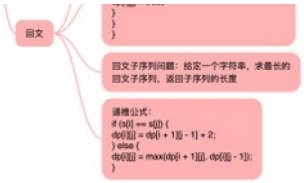



动态规划
股票问题
121.买卖股票的最佳时机——只能买卖一次
贪心算法：一次遍历，每次取最左的最小值，然后计算当前的最大利润
dp[i][0] 表示第 i 天不持有股票，手头的现金。
dp[i][1] 表示持有股票，手头的现金
1).今天不持有=max(前一天不持有，前一天持有+今天卖出的价格)
dp[i][0] = Math.max(dp[i - 1][0], dp[i - 1][1] + prices[i]);
2).今天持有=max(前一天持有，今天买入股票的价格)
dp[i][1] = Math.max(dp[i - 1][1], -prices[i]);
122.买卖股票的最佳时机2——可以买卖多次
和121不同之处在于，买入股票的时候，要考虑之前买卖股票所得利润
dp[i - 1][0]
dp[i][0] = Math.max(dp[i - 1][0], dp[i - 1][1] + prices[i]);
dp[i][1] = Math.max(dp[i - 1][1], dp[i - 1][0] - prices[i]);
(dp[i - 1][0] - prices[i] 体现了股票可以买卖多次)
123.买卖股票的最佳时机3——最多买卖两次
dp[i][1] = Math.max(dp[i - 1][1], -prices[i]); //第一次买入
dp[i][2] = Math.max(dp[i - 1][2], dp[i - 1][1] + prices[i]); //第一次卖出
dp[i][3] = Math.max(dp[i - 1][3], dp[i - 1][2] - prices[i]); //第二次买入
dp[i][4] = Math.max(dp[i - 1][4], dp[i - 1][3] + prices[i]); //第二次卖出
初始化：dp[0][1] = -prices[0]; dp[0][3] = -prices[0];
dp[i][0] : 表示没有操作，可以不用管
188.买卖股票的最佳时机4——最多买卖k次
int[][] dp = new int[prices.length][2 * k + 1];
dp[i][j] 表示第i天，第j次买入或卖出股票，手头的现金是多少。
(j 为奇数表示买入状态，偶数表示卖出状态)
309.最佳买卖股票时机含冷冻期
/* dp[i][j]，第i天状态为，所剩的最多现金为dp[i][j]。
* j的状态为：
* 0：持有股票后的最多现金
* 1：不持有股票（能购买）的最多现金
* 2：不持有股票（冷冻期）的最多现金
*/
代码详情见子主题
714.买卖股票的最佳时机含手续费
就是在122基础上，多了一个减去手续费的操作
子序列问题
子序列(不连续)
300.最长递增子序列
dp[i] 表示i之前，包括的最长递增子序列长度
for (int i = 1; i < nums.length; i++) {
for (int j = 0; j < i; j++) {
//注意这里不是要dp[i] 与 dp[j] + 1进行比较，而是我们要取dp[j] + 1的最大值。
if (nums[i] > nums[j]) dp[i] = Math.max(dp[i], dp[j] + 1);
}
if (dp[i] > result) result = dp[i]; //记录最大值
}
1.外层for循环确定dp[i]的值，内层for循环比较nums[i] 与 前面nums[j]:nums[0]~nums[i - 1],
如果nums[i] > nums[j],则在dp[j]基础上+1，通过比较
max(dp[i], dp[j] + 1),确定dp[i] (也就是取dp[j] + 1的最大值)
2.注意在外层for循环中，保留最长的递增序列，因为dp数组最后一位不一定是最长的
1143.最长公共子序列
dp[i][j] 表示 下标-1的text1 和 下标为j-1的text2最长公共子序列长度
text1和text2的中的字符相等:dp[i][j] = dp[i - 1][j - 1] + 1;
(相当于不考虑当前字符，考虑到双方当前字符的前一个字符为止,即dp[i - 1][j - 1]，然后+1)
如若不相等,由左或上推导得出，相当于删除text1/text2的当前字符，看看此时它们的最长公共子序列最大是多少，即dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
处理公共问题，或者两个序列/字符串比较，创建的dp[][]数组长度要+1
392.判断子序列
和1143几乎一模一样
1035.不相交的线
跟1143几乎一模一样
搞清楚dp[i]/dp[i][j]的定义，然后再举例推导dp数组
要知道dp[i]/dp[i][j]是从哪个方向推导得到,
就比较容易想到遍推公式了，接着要注意初始化和遍历
顺序的细节问题。
看题目是否需要一个res来记录最大值，因为dp数组的最后一位不一定是最大的结果
子序列(连续)
674.最长连续递增序列
连续问题，所以只能由前一个状态推导当前状态:dp[i] = dp[i - 1] + 1;
记得用一个变量保存dp[i]的最大值
718.最长重复子数组
dp[i][j] ：以下标 - 1为结尾的nums1，和以下标 - 1为结尾的nums2，
最长重复子数组长度为dp[i][j]
若匹配上，因为是连续问题，则dp[i][j],一定是由之前匹配上的结果
dp[i - 1][j - 1] + 1得到。即：dp[i][j] = dp[i - 1][j - 1] + 1
处理公共问题，或者两个序列/字符串比较，创建的dp[][]数组长度要+1
53. 最大子序和
dp[i] 表示下标为i的数组，最大子数组和为dp[i]
dp[i] = Math.max(nums[i], dp[i - 1] + nums[i]);
记得用result变量记录最大值，result初值为
nums[0](为了应对[-1, -2] 这种测试用例)
编辑距离
115.不同的子序列
dp[i][j]：以i-1为结尾的s子序列中出现以j-1为结尾的t的个数为dp[i][j]。
若匹配不上，dp[i][j] = dp[i - 1][j] (相当于s的这个当前字符没用，等价于不考虑它，把它删了，取到当前字符的前一个字符的状态dp[i - 1][j],转移给dp[i][j])
若匹配上，dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j];(等价于：把两个字符都删除一个的匹配个数+把s的当前字符删除得到的匹配个数)
举例：s="bagg",t = "bag",匹配到最后一位时，答案为2.
根据状态转移方程，s = bag, t = ba ,这是一种情况
s= bag, t=bag,也是一种情况，最后的结果就是2 = 1+1
583.两个字符串的删除操作
方法一：先求字符串的最长公共子序列，
然后两个字符串的长度减去最长公共子序列即可
dp[i][j]：以i-1为结尾的字符串word1，和以j-1位结尾的字符串word2，
最长连续公共子序列长度为dp[i][j]。
1).若匹配上：dp[i][j] = dp[i - 1][j - 1] + 1
2).若匹配不上：dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
方法二：
dp[i][j]：以i-1为结尾的字符串word1，和以j-1位结尾的字符串word2，
想要达到相等，所需要删除元素的最少次数。
1).若匹配上：dp[i][j] = dp[i - 1][j - 1]
2).若匹配不上：dp[i][j] = min(dp[i - 1][j - 1] + 2,
dp[i - 1][j] + 1, dp[i][j - 1] + 1);
72.编辑距离
注意！！！：此题删除和添加是等价操作，所以只需要考虑 替换 和 删除 两种操作即可
dp[i][j]:以下标i-1为结尾的word1, 以下标j-1为结尾的word2,想要达到相同,需要的编辑距离为dp[i][j]
1).若匹配上：dp[i][j] = dp[i - 1][j - 1] ,
2).若匹配不上：dp[i][j] = Math.min(dp[i - 1][j - 1] + 1, //替换字符
Math.min(dp[i - 1][j], dp[i][j - 1]) + 1); //删除字符
想清楚dp的含义！！！
一定要举例推导dp数组！！！
一边画图，一边对着dp[i][j]定义理解
分 匹配上 和 匹配不上两种情况
判断回文串
647.回文子串
方法一：暴力解法(代码见子主题)
方法二：动态规划
dp[i][j] 表示 区间[i, j]内的子串是否是回文串,若是则dp[i][j]为true
根据递推公式确定遍历顺序，从下到上，从左到右
如果匹配上s.charAt(i) == s.charAt(j) ，根据 j - i 的值做进一步判断(代码见子主图)。
此时要想到一个例子："a","aa","aaa"
516.最长回文子序列
dp[i][j] 表示在区间[i, j]内，最长的回文子序列长度为dp[i][j]
1).s[i]与s[j]相同，那么dp[i][j] = dp[i + 1][j - 1] + 2
相当于在内部区间[i + 1, j - 1]的最长回文子序列长度dp[i + 1][j - 1]的基础上又多了两个
2).若s[i]与s[j]不相同，看两边分别加入哪边，可以得到最长的回文子序列
即：dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);

注意对角线，dp[i][i]初始化为1

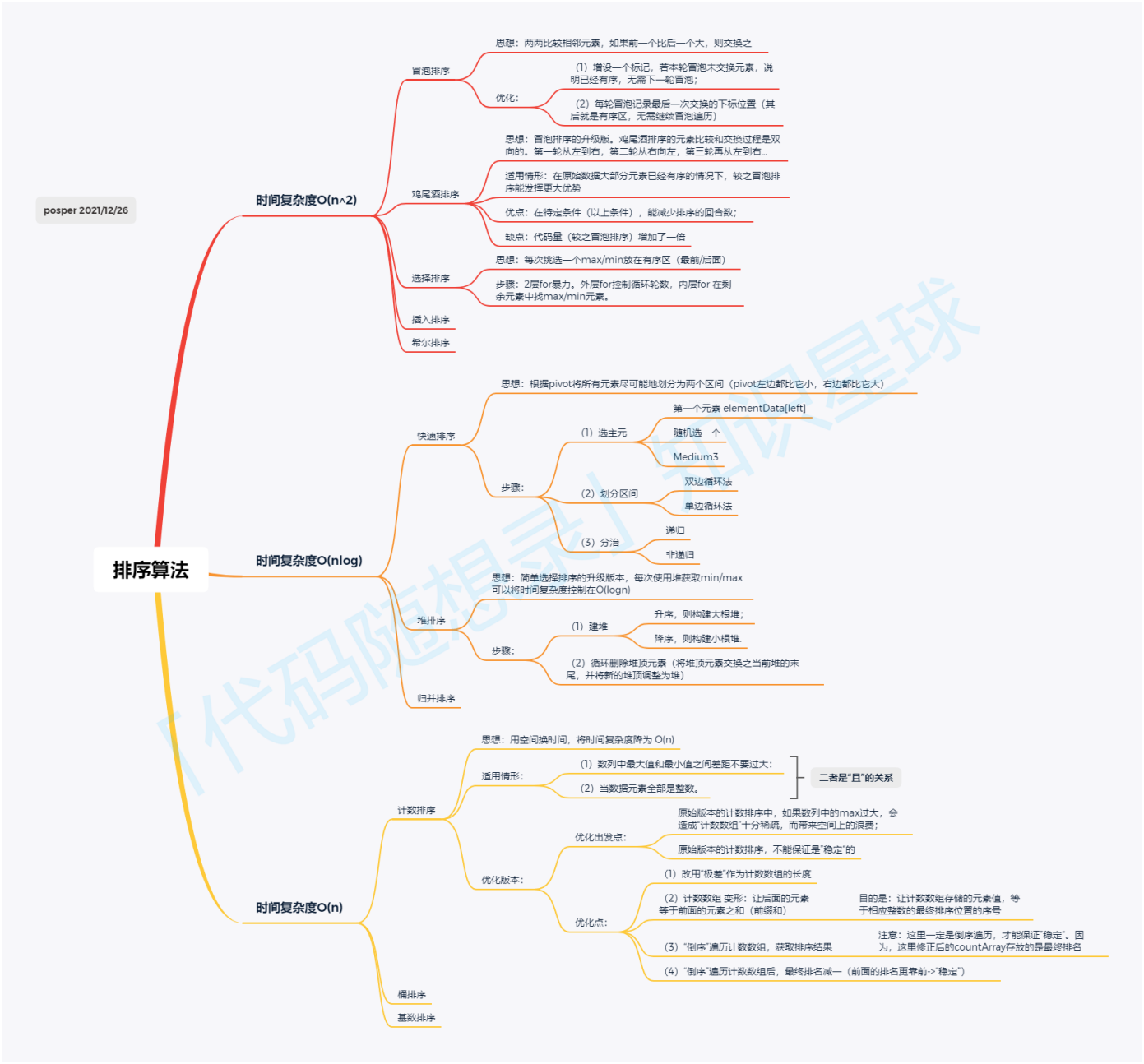
> 星球原帖：https://t.zsxq.com/14OVuRtMY



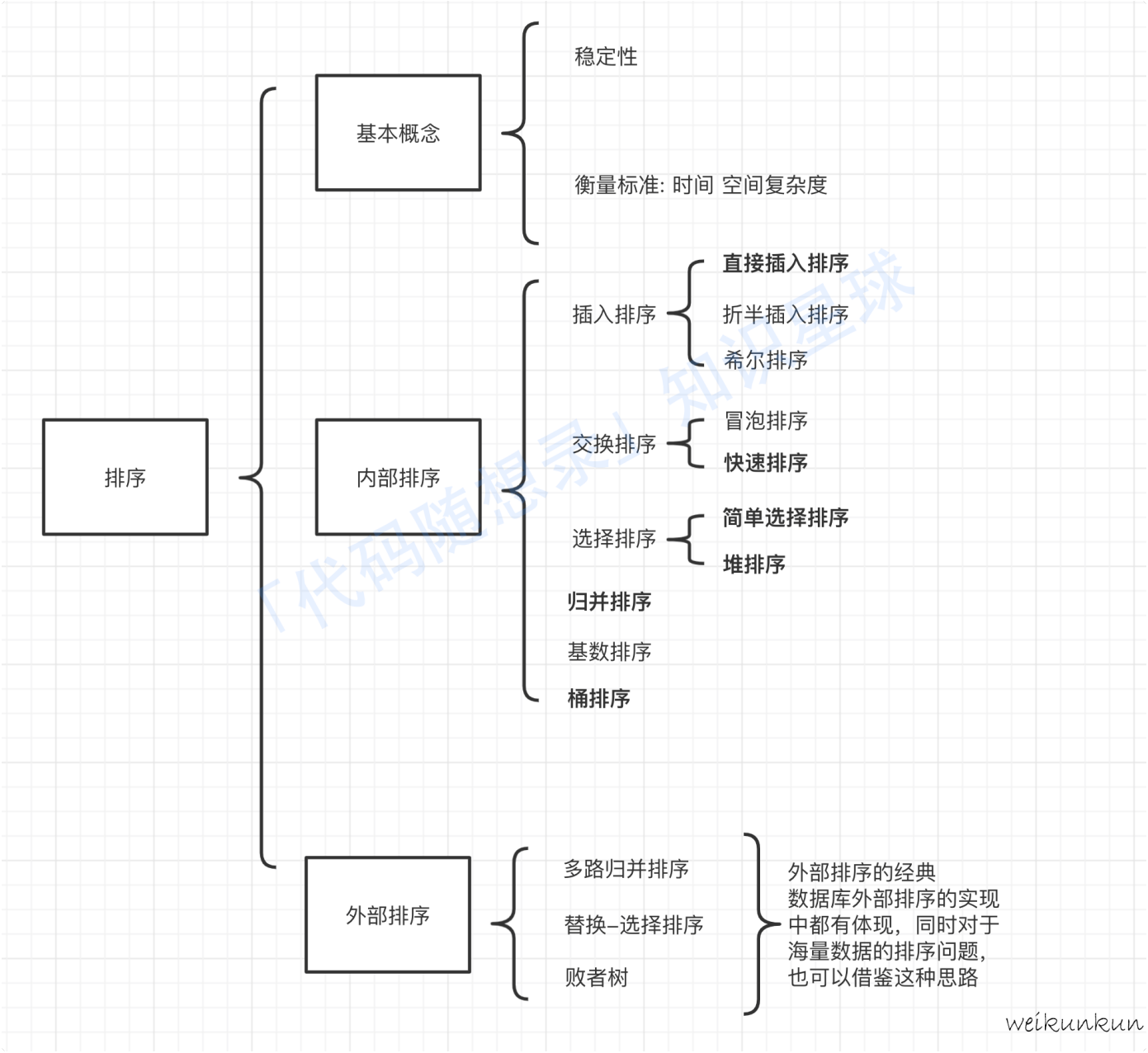


排序

原图链接：[「代码随想录」知识星球]



整体排序算法划分：



时间空间的分析（有些算法的不太确定）：

	时间复杂度	空间复杂度	是否为稳定排序	是否为原地排序
直接插入排序	$O(n^2)$	$O(1)$	✓	✓
折半插入排序	$O(n^2)$ (只优化了比较的过程)	$O(1)$	✓	✓
希尔排序	和增量有关，最坏$O(n^2)$	$O(1)$	✗	✓
冒泡排序	$O(n^2)$	$O(1)$	✓	✓
快速排序	$O(nlog_2n)$	$O(log_2n)$	✗	✓
简单选择排序	$O(n^2)$	$O(1)$	✗	✓
堆排序	$O(nlog_2n)$	$O(1)$	✗	✓
归并排序	$O(nlog_2n)$	$O(n)$	✓	✗
基数排序	$O(dn)d$是位数	$O(d)$	✓	✗
桶排序	$O(n+k)$ k为桶个数，n为元素数量	$O(n+k)$	✓	✗

weikunkun

排序算法时间空间复杂度分析

<table>
<tr><th></th><th>最好时间复杂度</th><th>最坏时间复杂度</th><th>平均时间复杂度</th><th>空间复杂度</th></tr>
<tr><td>冒泡排序</td><td rowspan="2">O(n)
[初始数组正序]</td><td rowspan="2">O(n²)
[初始数组倒序]</td><td rowspan="2">O(n²)</td><td rowspan="4">O(1)
[原地使用数组，没有使用额外的空间]</td></tr>
<tr><td>插入排序</td></tr>
<tr><td>选择排序</td><td colspan="3">O(n²)
[每一趟都选出最值,一趟为 O(n),一共跑 n 趟]</td></tr>
<tr><td>希尔排序</td><td>O(n)</td><td>O(n²)</td><td>O(n^1.3)</td></tr>
<tr><td>快速排序</td><td>O(nlog n)</td><td>O(n²)
[每一次取到的元素就是数组中最大值]</td><td>O(nlog n)</td><td>O(log n)
[递归栈的深度]</td></tr>
<tr><td>堆排序</td><td colspan="3">O(nlog n)
[堆排序时间复杂度分析:堆调整的时间复杂度为 O(log n),对 n 元素进行堆调整时间复杂度为 O(nlog n)]
[堆调整时间复杂度分析：根节点和排在最后的序号为 i 的叶子结点交换,堆调整的操作次数=树的深度=log n]</td><td>O(1)
[原地使用数组，没有使用额外的空间]</td></tr>
<tr><td>归并排序</td><td colspan="3">O(nlog n)
划分区间的时间复杂度为: O(log n)
归并 n 层的时间复杂度为: O(nlog n)
[归并每一层的时间复杂度为:O(n)
归并的层数为： log n]
总时间复杂度:O(nlog n)+O(log n)=O(nlog n)</td><td>O(n+log n)
[log n：递归栈的深度
n ：辅助存储结果的数组的长度]</td></tr>
<tr><td>计数排序</td><td colspan="3">O(n+k)
[k 为辅助桶的数据范围(数组最大值-最小值)]
求最大最小值:O(n)
遍历装入桶:O(n)
循环输出元素:最坏为 O(k)</td><td>O(n+k)
n:保存输出结果的数组
k:辅助桶数组
将输出结果直接覆盖到原数组，空间复杂度可降为 O(k)</td></tr>
<tr><td>桶排序</td><td colspan="3">O(n+k)</td><td>O(k)</td></tr>
<tr><td>基数排序</td><td colspan="4">O(n*m)</td></tr>
</table>